VisionGuard

AI-generated and real-labeled image classification

Submission	Value
Name	Syed Muhammad Hanzala
Roll number / project	23L-0907 / 23L-0907_A2_Ops
Prepared	25 October 2026, 00:56 Asia/Karachi
Graded source revision	90b93195d46d58ecfcae174062cf6f5fef8c28bf
GitHub	https://github.com/Hanzala-Shadow/23L-0907_A2_Ops
DagsHub	https://dagshub.com/Hanzala-Shadow/23L-0907_A2_Ops
Live model API	https://23l-0907-a2-ops-api.vercel.app/predict
Public readiness	https://23l-0907-a2-ops-api.vercel.app/health
Local Flask UI	http://127.0.0.1:5000/

The source revision above identifies the code and evidence used for this report. The PDF itself is added in a following documentation-only commit, avoiding a self-referential commit hash.

Assistance disclosure

OpenAI Codex assisted with environment setup, implementation, debugging, tests, experiment execution, deployment, and drafting this report. Official TensorFlow/Keras, MLflow, DVC, Docker, GitHub, Vercel and Postman documentation informed the tool configuration. The saved scripts, run receipts and logs provide the evidence behind the results. No claim is made that this work was completed without assistance.

Both repositories remain private. The student will share grading access separately. The prediction endpoint requires its server-side API key; no credential is printed in this report.

Project outcome and reading guide

The system audits and versions the Kaggle data, trains a custom CNN and MobileNetV2, compares eleven candidates, locks a validation-selected threshold, and evaluates two locked models once on test. The champion is MLflow registry version 2. A local Flask interface calls its LiteRT export on Vercel, then writes successful and failed attempts to the existing PostgreSQL database.

Task	Evidence in this report
1	Environment, inventory, EDA, duplicate policy, deterministic splits, DVC toy restore
2	Input pipeline, CNN layers, two transfer phases, discipline, test results, ten errors, tuner
3	Local and remote toy runs, manual/autolog tracking, artifacts, comparison, registry
4-5	Inference lifetime, upload validation, UI, configuration, PostgreSQL trail
6	Warm-up, image design, Compose, size optimization, recovery, red/green CI
7-8	Cloud deployment steps, Postman from another network, local API integration

Task 7's direct instruction and Task 8 require localhost Flask calling a cloud model API; this is the implemented arrangement. The example pathway table describes other layouts too. Vercel's DNS-safe resource name is 23l-0907-a2-ops-api; the GitHub, DagsHub, MLflow and Compose project identity is 23L-0907_A2_Ops. The Docker and DVC toy exercises were completed during the final audit rather than before the main implementation; their timestamps are retained.

Optional Grad-CAM bonus work was not attempted. No training, threshold selection, conversion or deployment check reuses the held-out test set after the recorded final evaluation. Screenshots are genuine browser captures; transcript screenshots are explicitly labeled recorded command output.

Task 1 - Data understanding and versioning

1.1 Isolated CPU environment

The Python virtual environment is .venv. TensorFlow CPU 2.19.1, Keras 3.15.1, MLflow 2.22.5 and DVC 3.67.1 are pinned with the other dependencies. GPU discovery returned an empty list. CPU thread counts are bounded for this machine. The environment smoke check trained synthetic tensors and verified Keras/MLflow save-load, Flask and SQLAlchemy before downloading the assignment data. (Figure 1.)

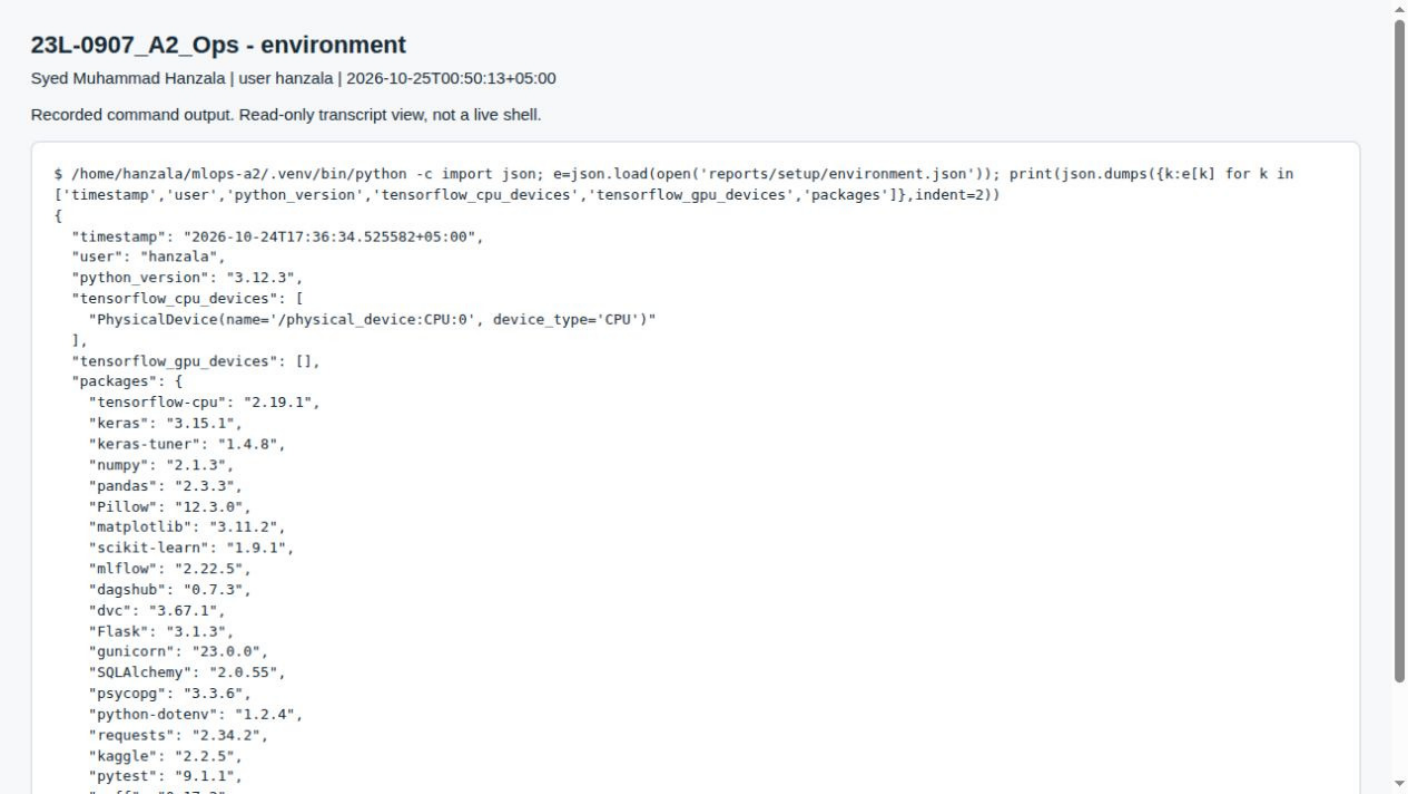


Figure 1. Recorded environment command output with project, user, time, versions and GPU result.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements/locked.txt
.venv/bin/python scripts/check_environment.py --smoke
.venv/bin/python scripts/check_disk.py --minimum-free-gib 2
```

venv isolates packages; the lock records exact versions; the smoke flag exercises functionality; the disk check stops unsafe work. With the environment, raw images and DVC cache retained, the reserve is checked before training, artifact work and uploads. No large local rebuild was needed for submission.

1.2 Audit and reproducible splits

Folder label	Files	Corrupt	Width min/med/max	Height min/med/max	Bytes min/med/max
ai_generated	539	0	225 / 1024 / 6000	135 / 800 / 4001	11738 / 199760 / 5626183
real	436	0	183 / 920.0 / 8495	158 / 720.0 / 7091	11311 / 144840.0 / 10195352

The custom walker records class, bytes, dimensions, colour mode, decoded format and SHA-256 for every file in reports/data/image_manifest.csv. All 975 files decode. One exact duplicate pair has the same class: image33.jpg and image34.jpg. The raw source is preserved, while split preparation excludes image34.jpg. There are 974 unique hashes and no conflicting exact-hash labels.

Split	AI-generated	Real-labeled	Total
Train	376	305	681
Validation	81	65	146
Test	81	66	147

A SHA-256 ranking of seed 42 and each content hash makes the stratified 70/15/15 lists independent of input row order. Largest-remainder allocation determines integer counts. Checks reject overlapping paths or hashes and verify source bytes. A forced DVC reproduction produced identical lists. Exact hashes do not detect differently encoded near-duplicates or establish true image provenance.

1.3 Visual EDA

The grid shows the collection includes photographs, paintings, composites and web graphics. Folder labels are the task target, not independently verified origin. This visual diversity is a likely source of errors and limits claims about real-world provenance. (Figure 2.)

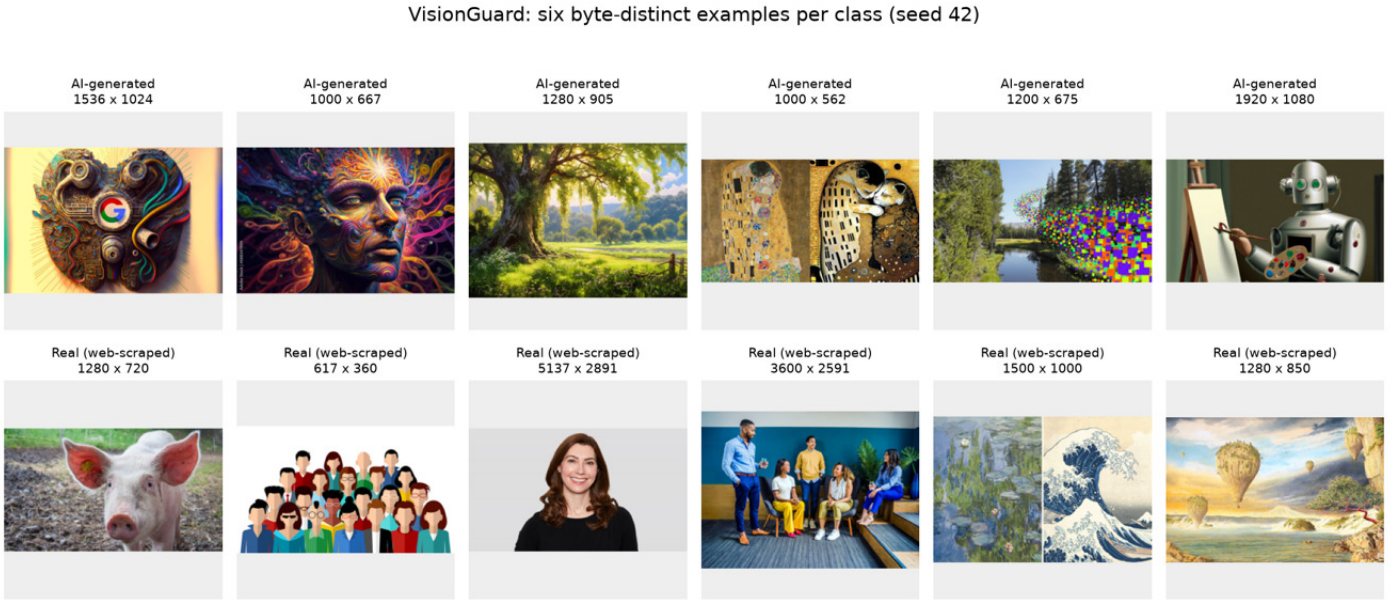


Figure 2. Seeded samples per folder class; source labels are not verified provenance.

Both classes have varied widths, with some very large files. AI-labeled widths cluster near common generation sizes, while real-labeled widths are also heterogeneous. Resizing is necessary for bounded CPU cost. (Figure 3.)

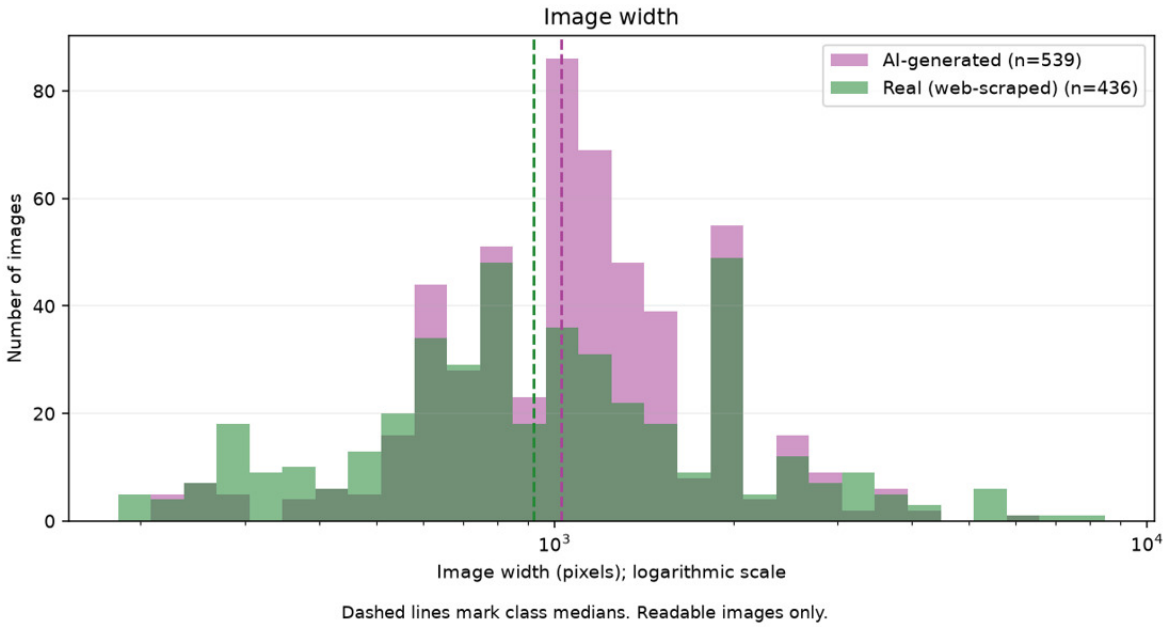


Figure 3. Per-class width distributions from the complete audit.

Image heights vary strongly and include portrait and landscape examples. A square training tensor would distort subjects if stretched. White letterboxing preserves the aspect ratio while fixing tensor dimensions. (Figure 4.)

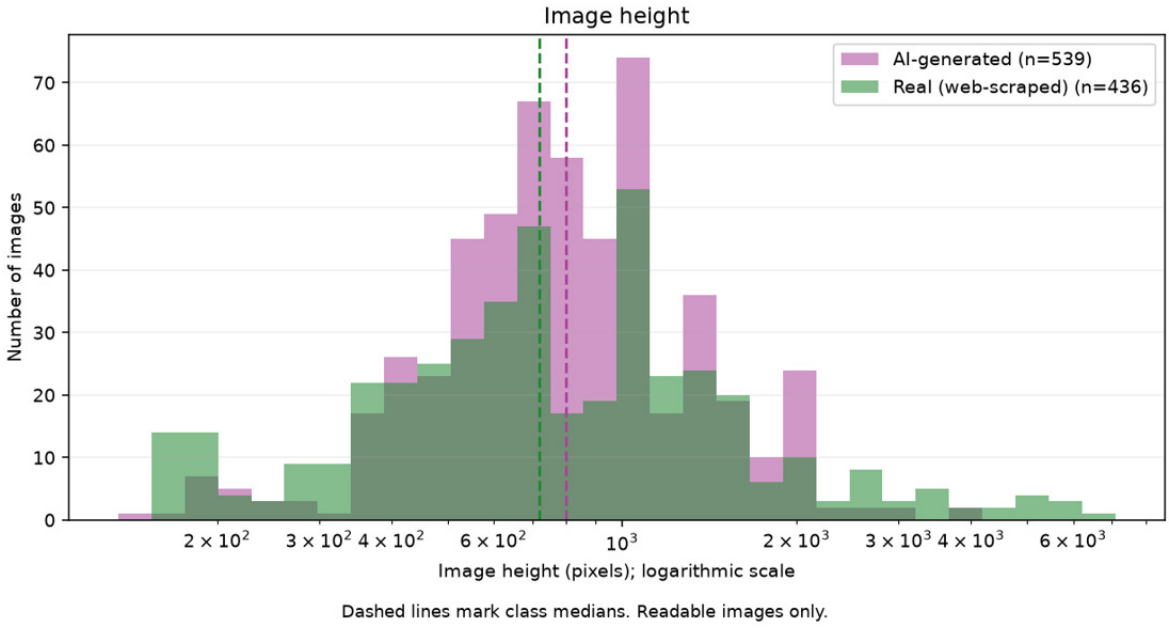


Figure 4. Per-class height distributions from the complete audit.

File sizes have a long tail and depend on compression and content as well as dimensions. A small compressed file can still decode to many pixels. Serving therefore checks both byte count and pixel count. (Figure 5.)

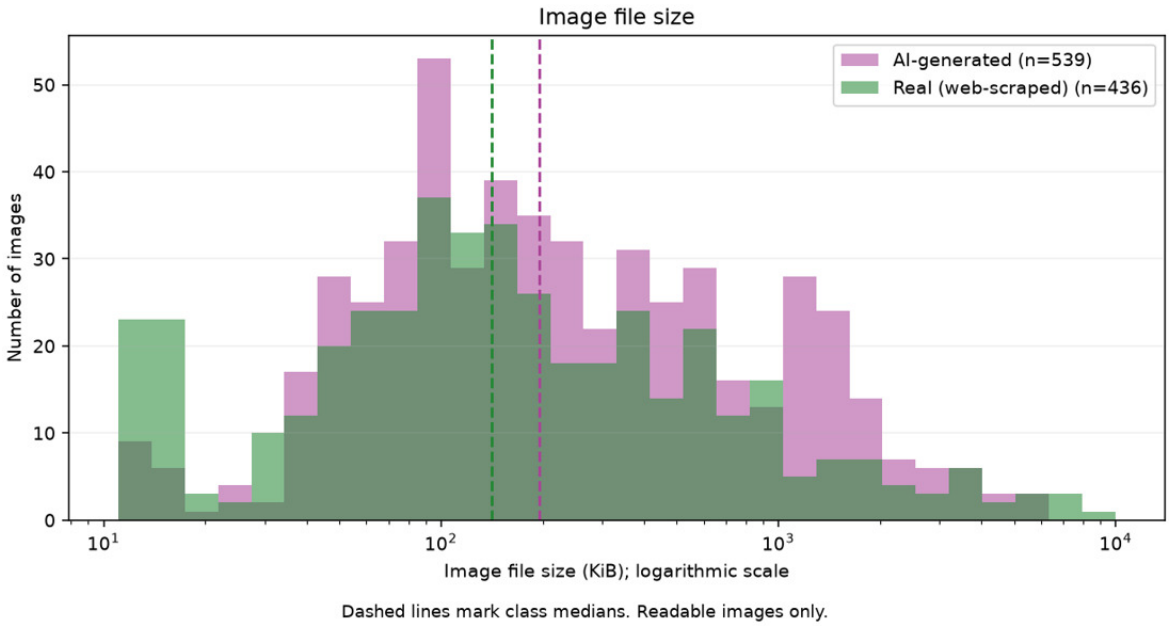


Figure 5. Per-class file size bytes distributions from the complete audit.

1.4 DVC toy lab and project storage

The separate tiny CSV lab initialized Git and DVC in an empty folder, committed a two-row CSV pointer, added a third row, and committed the second pointer. Both versions were pushed on the dvc-lab branch of the existing private project. A local folder outside that toy repository is its remote. Returning to the first commit restored the older CSV. Deleting both its workspace file and its toy cache, then running dvc pull, restored exactly the old bytes. Production images and cache were untouched.

Version	Git commit
Initial	0a21a67dd5c2aec14e7fefdaf8230a37f6dc9fa6
Updated	d4e9e1e4bc6787180e8d0cb80e22cc027531c4f6

```
outs:
- md5: 46bc404c9689a8c77064cf81aa0f39d2
size: 48
hash: md5
path: examples.csv
```

The pointer stores a checksum, byte size and relative output path. Git versions this small text; DVC stores large content separately by checksum. The real project uses DagsHub's S3-compatible DVC store. A previous empty-cache restore verified all 975 source hashes and five split files. Models are also DVC-versioned. Credentials stay in ignored .dvc/config.local, never shared configuration. (Figure 6.)



Figure 6. Recorded DVC two-version history, older-version checkout and clean-cache pull verification.

Task 2 - Model development

2.1 Input pipeline and augmentation

Split CSVs supply paths, labels and original hashes. The shared Pillow decoder applies EXIF orientation, converts to RGB, composites transparency on white, then bilinearly resizes into a centered 128 x 128 letterbox. tf.numpy_function bridges that decoder into tf.data; shape is fixed to 128 x 128 x 3. Decoded uint8 tensors are cached in RAM, training is shuffled with seed 42, batches contain 16 images and one batch is prefetched. Decoding uses one worker and the two caches use roughly 39 MiB. No resized image dataset is written to disk.

128 pixels bounds CPU time but loses fine artifacts. Aspect ratio is preserved through padding. Training-only Keras layers flip horizontally, rotate up to 18 degrees and zoom up to 10%. They are inactive for validation and inference. Normalization is saved inside each model. (Figure 7.)

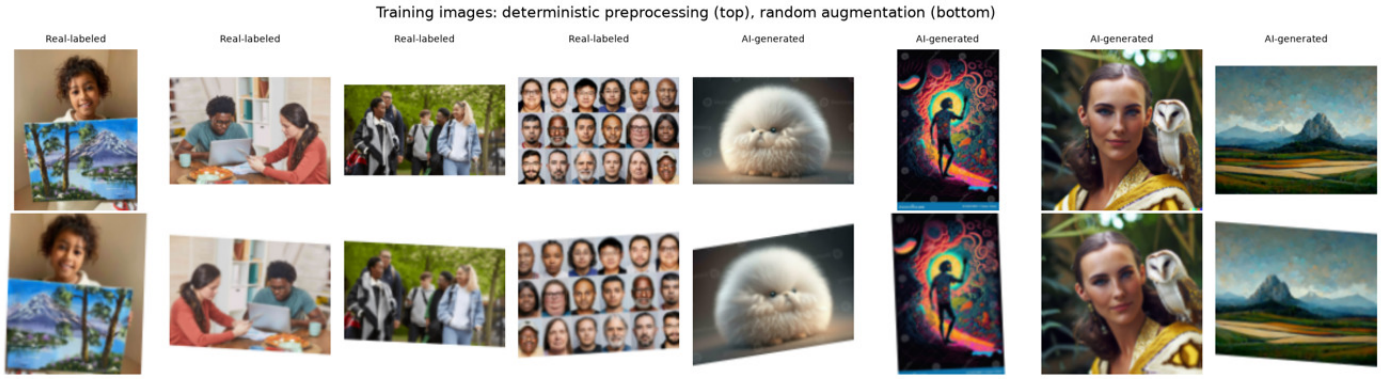


Figure 7. Original and augmented training images; no validation or test augmentation.

2.2 Custom CNN baseline

Three convolution blocks have 16, 32 and 64 filters. A convolution learns local spatial patterns; BatchNorm stabilizes activations; ReLU adds nonlinearity; max pooling reduces spatial cost. Biases are omitted before BatchNorm. Global average pooling replaces an expensive flatten layer. A 64-unit head and dropout 0.3 reduce capacity; one sigmoid estimates the AI-generated probability. The model has 28,145 total parameters. Binary cross-entropy matches the target; Adam starts at 0.001. Accuracy, precision, recall and ROC-AUC make different errors visible. (Figure 8.)

Layer	Type	Output shape	Parameters
rgb_pixels	InputLayer	(None, 128, 128, 3)	0
rescale	Rescaling	(None, 128, 128, 3)	0
training_augmentation	Sequential	(None, 128, 128, 3)	0
conv_1	Conv2D	(None, 128, 128, 16)	432
batch_norm_1	BatchNormalization	(None, 128, 128, 16)	64
relu_1	ReLU	(None, 128, 128, 16)	0
pool_1	MaxPooling2D	(None, 64, 64, 16)	0

Layer	Type	Output shape	Parameters
conv_2	Conv2D	(None, 64, 64, 32)	4608
batch_norm_2	BatchNormalization	(None, 64, 64, 32)	128
relu_2	ReLU	(None, 64, 64, 32)	0
pool_2	MaxPooling2D	(None, 32, 32, 32)	0
conv_3	Conv2D	(None, 32, 32, 64)	18432
batch_norm_3	BatchNormalization	(None, 32, 32, 64)	256
relu_3	ReLU	(None, 32, 32, 64)	0
pool_3	MaxPooling2D	(None, 16, 16, 64)	0
global_pool	GlobalAveragePooling2D	(None, 64)	0
head	Dense	(None, 64)	4160
dropout	Dropout	(None, 64)	0
ai_probability	Dense	(None, 1)	65

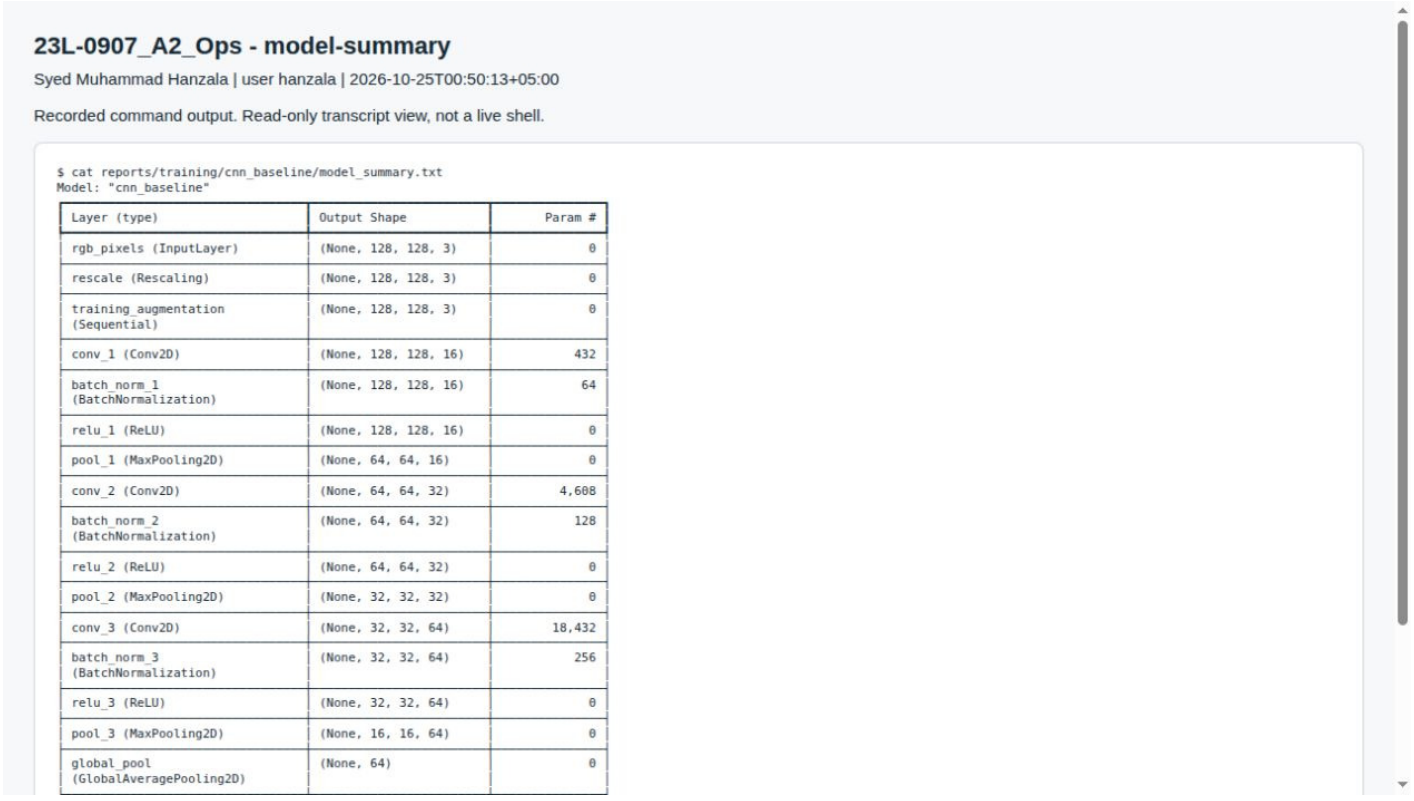


Figure 8. Recorded Keras summary for the custom CNN, including parameter totals.

2.3 Frozen backbone and fine-tuning

ImageNet MobileNetV2 is loaded without its original classifier. Its head uses global average pooling, Dense(64), dropout 0.3 and a sigmoid. Its expected normalization is pixel / 127.5 - 1, saved as a

Rescaling layer; the CNN instead uses pixel / 255. Passing already normalized pixels to these models would apply normalization twice and break the training-serving contract.

The frozen stage trains only 82,049 head parameters at learning rate 0.001. The second stage starts from the best frozen checkpoint and recompiles Adam at 0.00001. Only block_16_expand, block_16_depthwise and block_16_project become trainable: 551,489 trainable parameters including the head. Earlier blocks, Conv_1 and every BatchNorm layer remain frozen. Calling the backbone with training=False preserves moving statistics while selected convolution weights receive gradients. Weight hashes verified those boundaries. Fine-tuning worsened validation loss, so it was not assumed superior. (Figures 9, 10.)

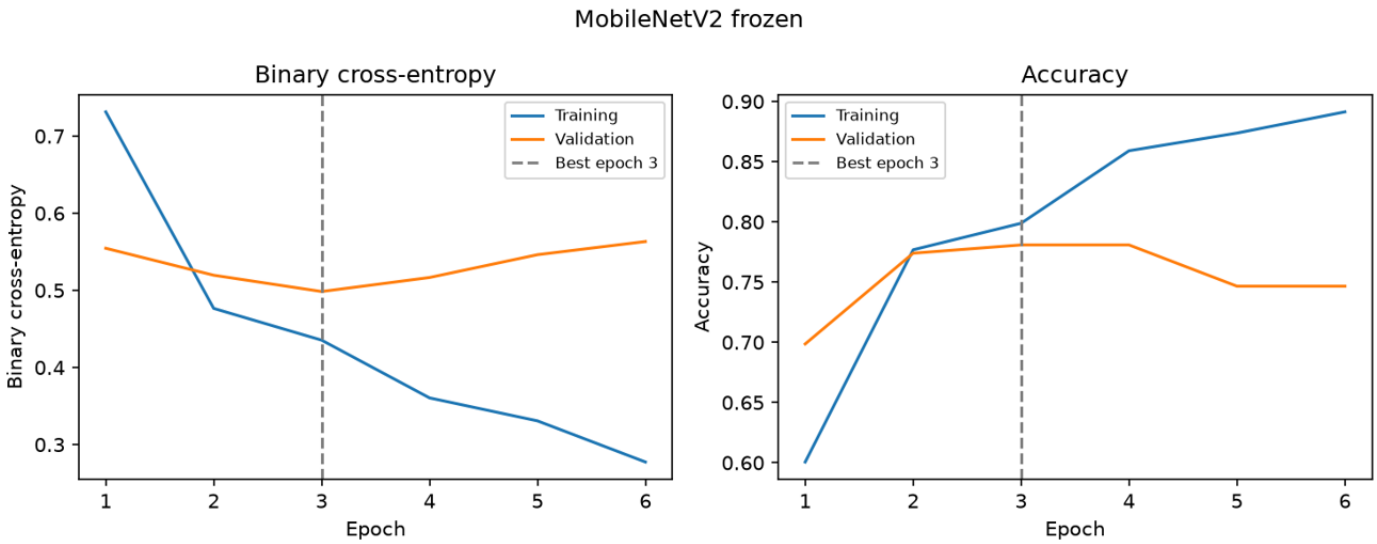


Figure 9. Frozen-head loss and accuracy; best validation-loss epoch is marked.

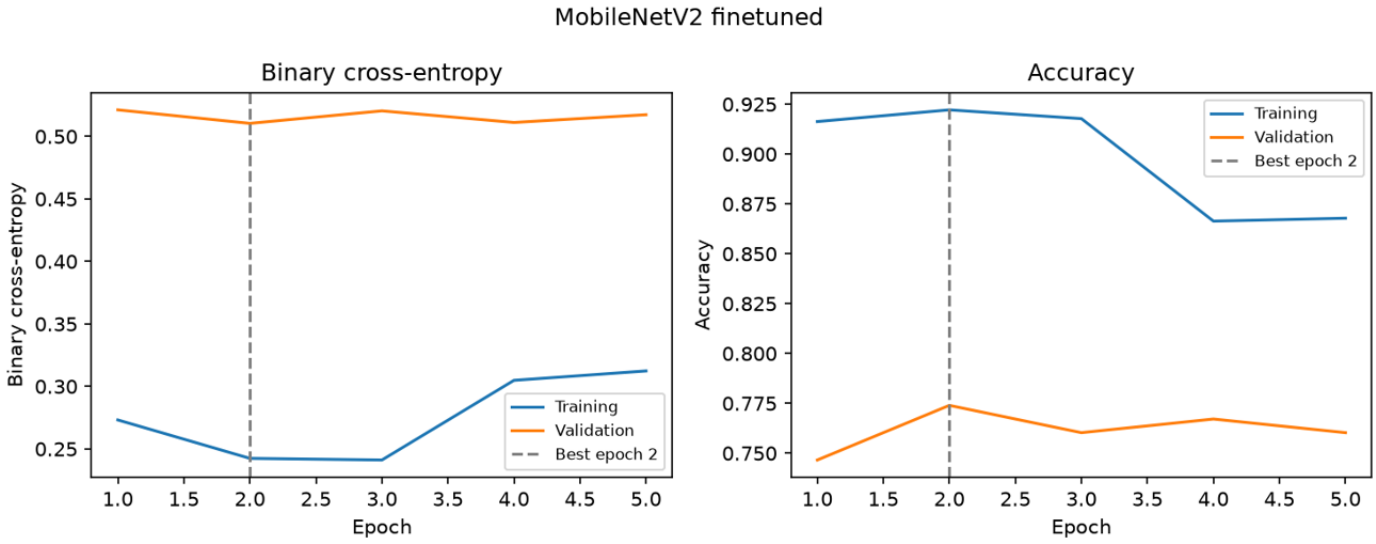


Figure 10. Last-block fine-tuning curves; earlier weights and BatchNorm remained frozen.

2.4 Training discipline

Seed 42 fixes Python, NumPy and TensorFlow random choices. Deterministic TensorFlow operations, two intra-op threads, one inter-op thread, disabled XLA and disabled oneDNN reduce variation. Exact

cross-hardware equivalence is not guaranteed. Every stage checkpoints minimum validation loss, stops after no improvement and reduces the learning rate on a plateau. Only the best checkpoint is used for validation predictions; the final epoch is not silently substituted. (Figure 11.)

Run	Best epoch	Epochs completed	Best validation loss
cnn_baseline	1	6	0.70121
mobilenetv2_frozen	3	6	0.49845
mobilenetv2_finetuned	2	5	0.51045

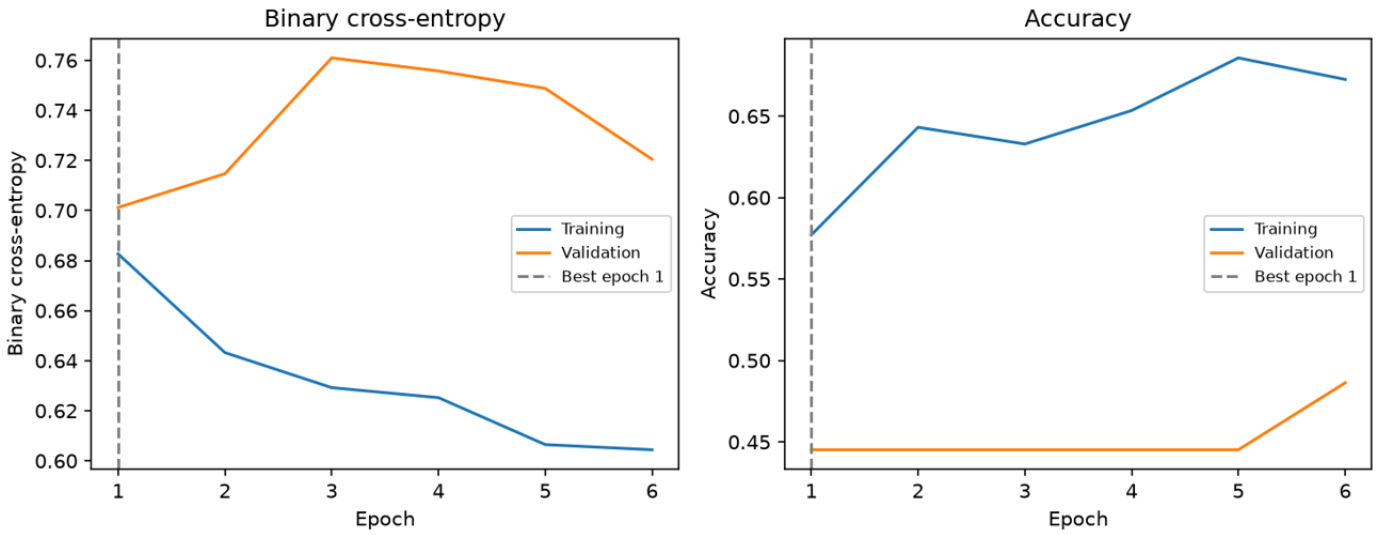


Figure 11. CNN early stopping after six epochs; the best checkpoint was epoch 1.

2.5 Locked threshold and one test evaluation

The champion was selected from eleven candidates by minimum validation cross-entropy. Its threshold 0.6196606457233429 minimizes 2 x false positives + false negatives on validation. A false accusation that a real-labeled image is fake is assigned twice the cost of missing an AI image. This is a stated product assumption, not an empirically measured harm ratio. Model and threshold were locked before test evaluation; the CNN received its own validation-selected threshold.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
cnn_baseline	0.4014	0.2308	0.0370	0.0638	0.4669
champion	0.7211	0.7857	0.6790	0.7285	0.8161

Both locked models received exactly one inference pass on 147 test images (66 real-labeled, 81 AI-generated). Subsequent figures use the saved probability arrays. Identity receipts bind the model hash, split manifest, threshold and selection-plan hash; a durable ledger refuses another test pass. The champion confusion matrix is TN=51, FP=15, FN=26, TP=55. The CNN is weak (TN=56, FP=10, FN=78, TP=3), and that result is retained rather than retuned on test. (Figures 12, 13, 14, 15.)

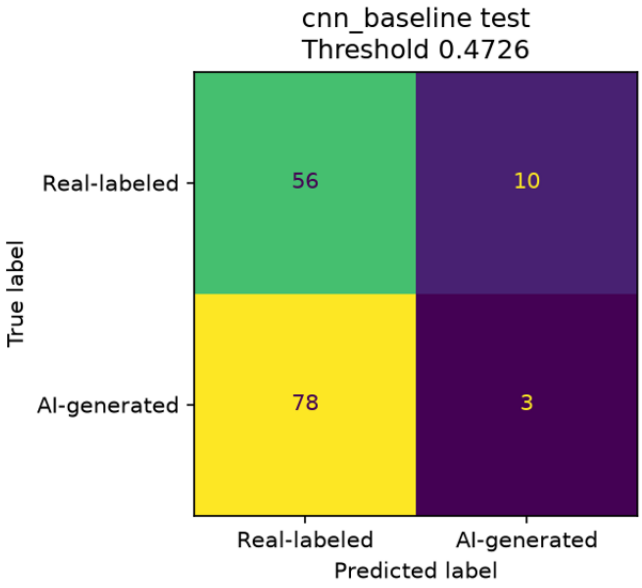


Figure 12. cnn_baseline: confusion matrix from the single saved test pass.

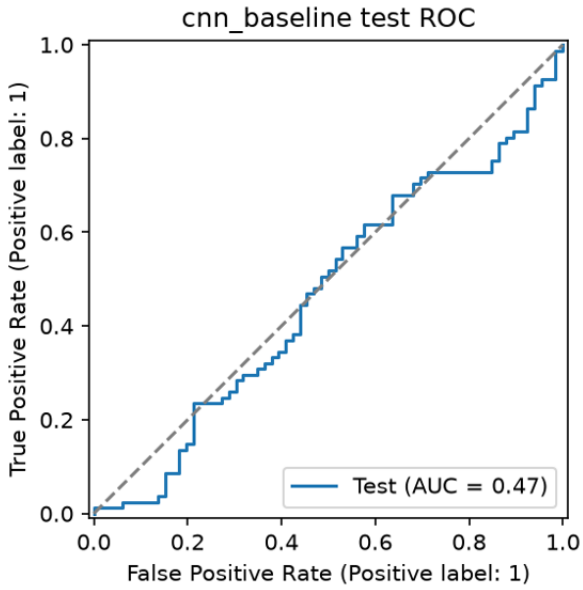


Figure 13. cnn_baseline: test ROC using the same saved probabilities.

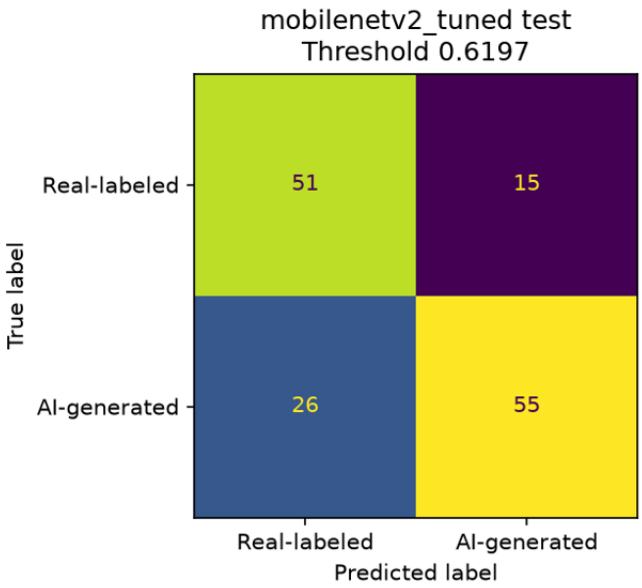


Figure 14. champion: confusion matrix from the single saved test pass.

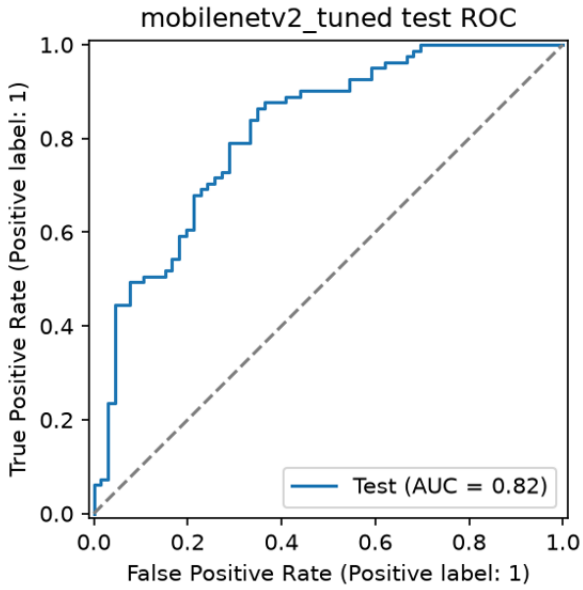


Figure 15. champion: test ROC using the same saved probabilities.

Champion accuracy dropped from 80.82% on validation to 72.11% on test. The data are small and varied, and validation was repeatedly reused during search. Neither these metrics nor a confident single prediction establish image origin. The observed test result did not change the locked model. (Figure 16.)

2.6 Ten misclassified examples

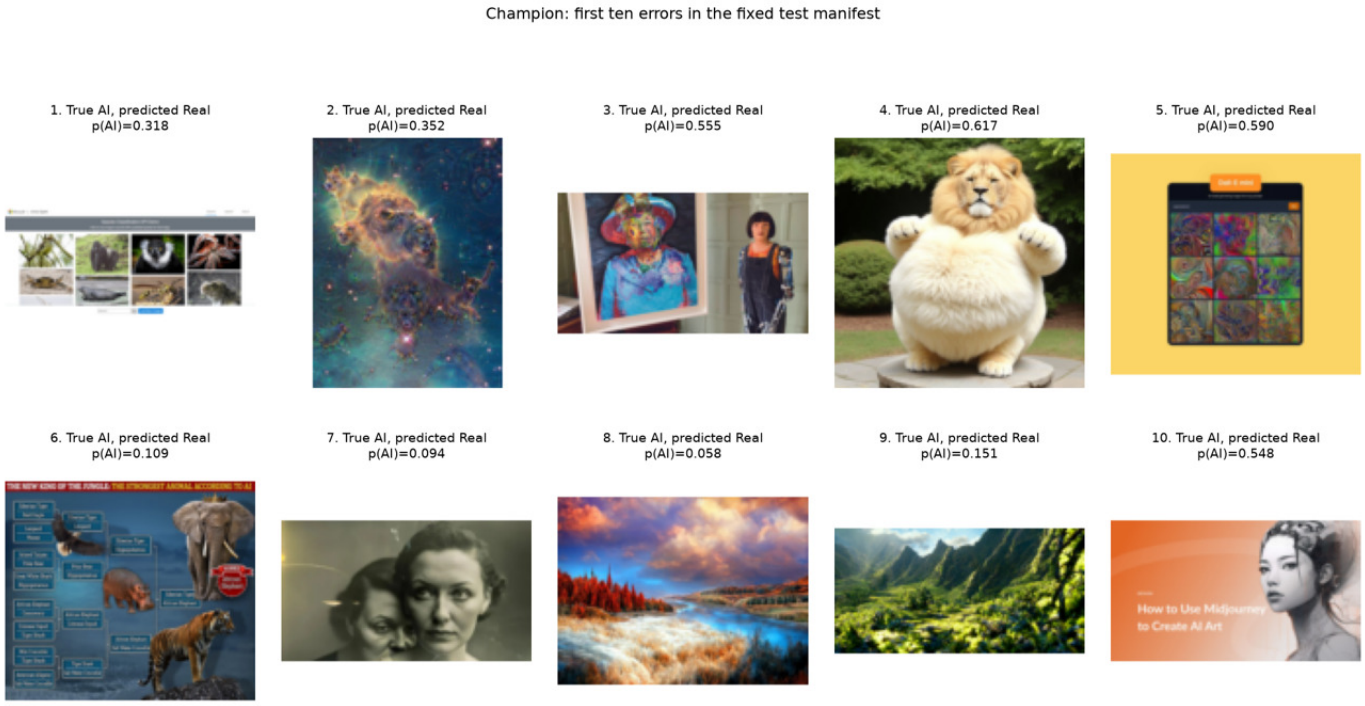


Figure 16. The first ten errors in fixed test-manifest order, rendered from the saved evaluation selection.

All ten examples are false negatives because AI-labeled paths occur first in the fixed manifest; this is not a representative sample of all error types. The full evaluation also contains 15 false positives.

Example	Visual hypothesis
1	A multi-photo web screenshot may favor photo and page-layout cues.
2	A nebula-like painterly texture overlaps with real-labeled artwork.
3	Photographic context around framed artwork may dominate its small generated region.
4	Fur and garden textures look familiar; probability 0.617 is just below the 0.620 threshold.
5	A product graphic has a large background and small artworks; resizing suppresses detail.
6	An infographic mixes text and cutouts, unlike a single isolated generated image.
7	A grainy monochrome portrait may evoke real photographic cues.
8	A saturated landscape has ordinary scene structure shared with real-labeled artwork.
9	A plausible mountain scene may favor semantic content over synthesis artifacts.
10	A promotional banner combines text and a small portrait; its 0.548 score misses the cutoff.

These observations are hypotheses, not causal attribution results. They motivate a future independent data review and are not used to alter the current test evaluation.

2.7 Keras Tuner tool lab and eight project trials

The built-in MNIST toy lab uses 6,000 training and 1,000 validation examples drawn only from MNIST's training partition. RandomSearch tests two learning rates and two hidden sizes for three epochs. The best setting is learning rate 0.01 and 64 units, validation loss 0.24696. Its four runs are logged separately on DagsHub.

Toy trial	Learning rate	Units	Validation loss	Accuracy
0	0.01	32	0.3145	0.9000
1	0.001	32	0.4416	0.8840
2	0.01	64	0.2470	0.9150
3	0.001	64	0.3765	0.8960

Trial	Learning rate	Dropout	Unfrozen convs	Val. loss	Best epoch
0	0.0001	0.2	1	0.51384	4
1	1e-05	0.5	1	0.50332	2
2	1e-05	0.5	3	0.50416	2
3	0.0001	0.2	3	0.54409	4
4	1e-05	0.2	3	0.51258	2
5	1e-05	0.2	1	0.51104	2
6	0.0001	0.5	1	0.49604	4
7	0.0001	0.5	3	0.48514	3

The project search covers the complete two-level factorial grid: rates 0.00001/0.0001, dropout 0.2/0.5 and one/three unfrozen final-block convolution layers. All trials start from the same frozen checkpoint. Trial 7 wins at 0.0001, dropout 0.5 and three unfrozen convolutions, validation loss 0.485135. Dropout has the largest marginal effect: mean validation loss falls by 0.02322 at 0.5. Learning rate and unfrozen count have smaller average effects, but interactions and a small sample limit causal claims. Selecting repeatedly on validation can overfit that set, even with early stopping.

Evidence: docs/tuning.md and reports/training/keras_tuner/summary.json. No search used test images.

Task 3 - MLflow and DagsHub

3.1 Local MLflow tool lab

A 16-unit Keras network trained on 1,200 MNIST training examples with 200 validation examples for two epochs. Two local runs differ only in Adam rate: 0.001 versus 0.01. Each logs parameters, epoch metrics, a loss plot and the model. The local SQLite-backed MLflow UI shows both runs and their comparison. At this tiny budget, validation accuracy is 0.54 versus 0.80, respectively; this toy result is not comparable with VisionGuard. (Figures 17, 18.)

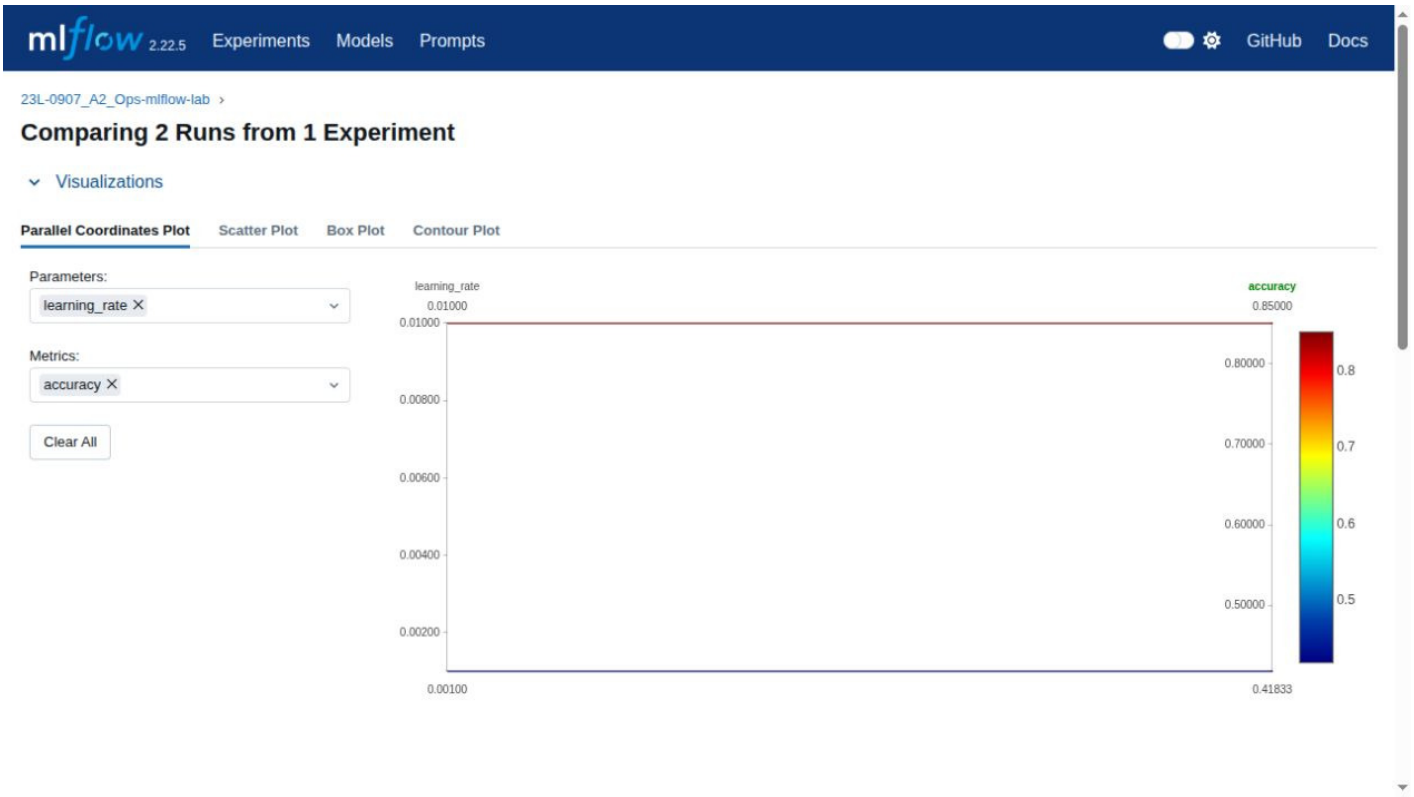


Figure 17. Local MLflow UI comparing the two real MNIST runs.

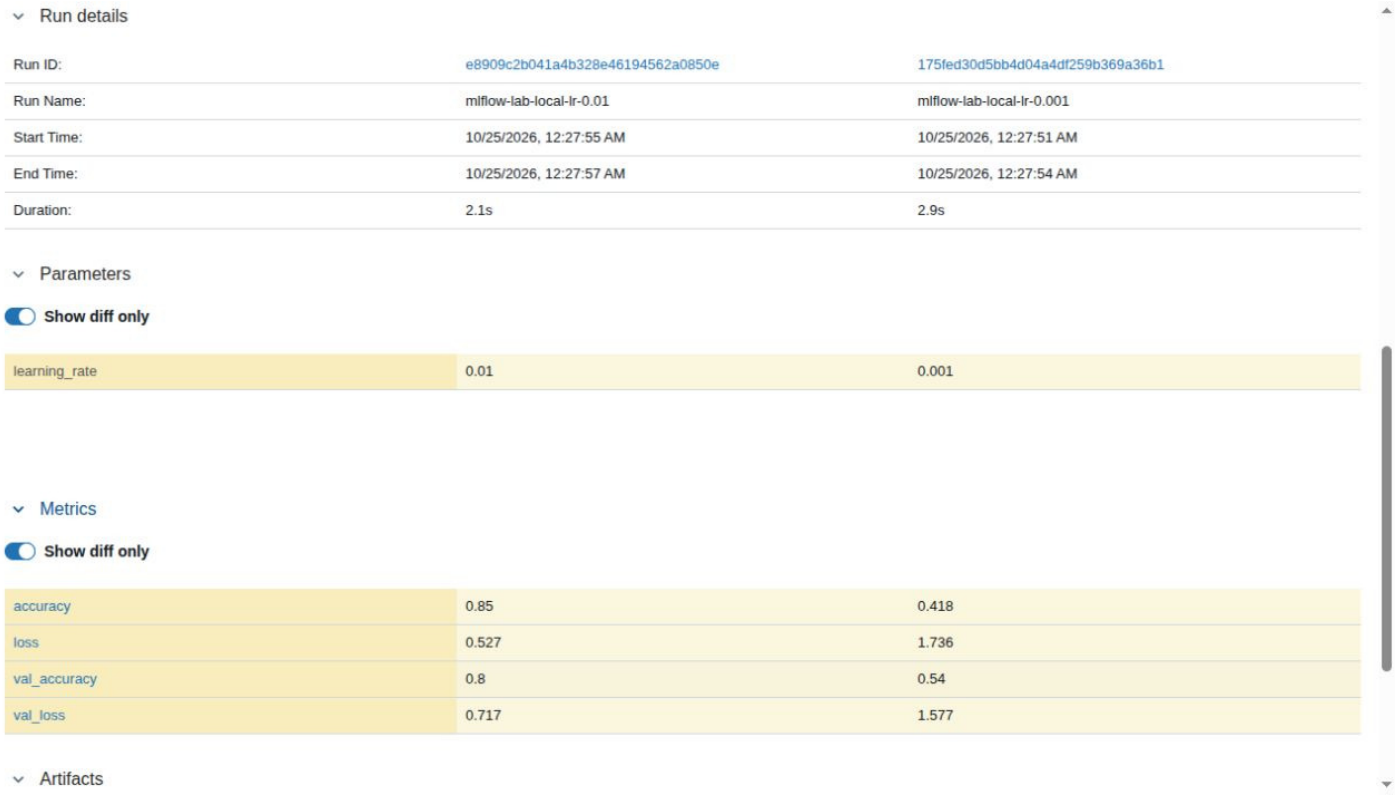


Figure 18. Local comparison view with run times, learning rates and epoch-final metrics.

Term	Meaning in this project
Experiment	A named workspace grouping related runs.
Run	One recorded training attempt with an identity and lifecycle.
Parameter	A configuration value, such as learning rate, fixed for the attempt.
Metric	A measured numeric result, often recorded at each epoch.
Artifact	A saved file such as a model, plot, CSV or package list.
Tag	Searchable context, such as source commit, seed or logging method.

3.2 DagsHub connection and toy repeat

The private DagsHub project mirrors GitHub. Its MLflow tracking URI is https://dagshub.com/Hanzala-Shadow/23L-0907_A2_Ops.mlflow. The client reads credentials from environment variables populated by ignored .env. git check-ignore confirms .env, .env.cloud and .dvc/config.local stay untracked. A 0.001-rate toy run was repeated on DagsHub, with two epoch records, its plot and model independently verified through the tracking API. (Figures 19, 20.)

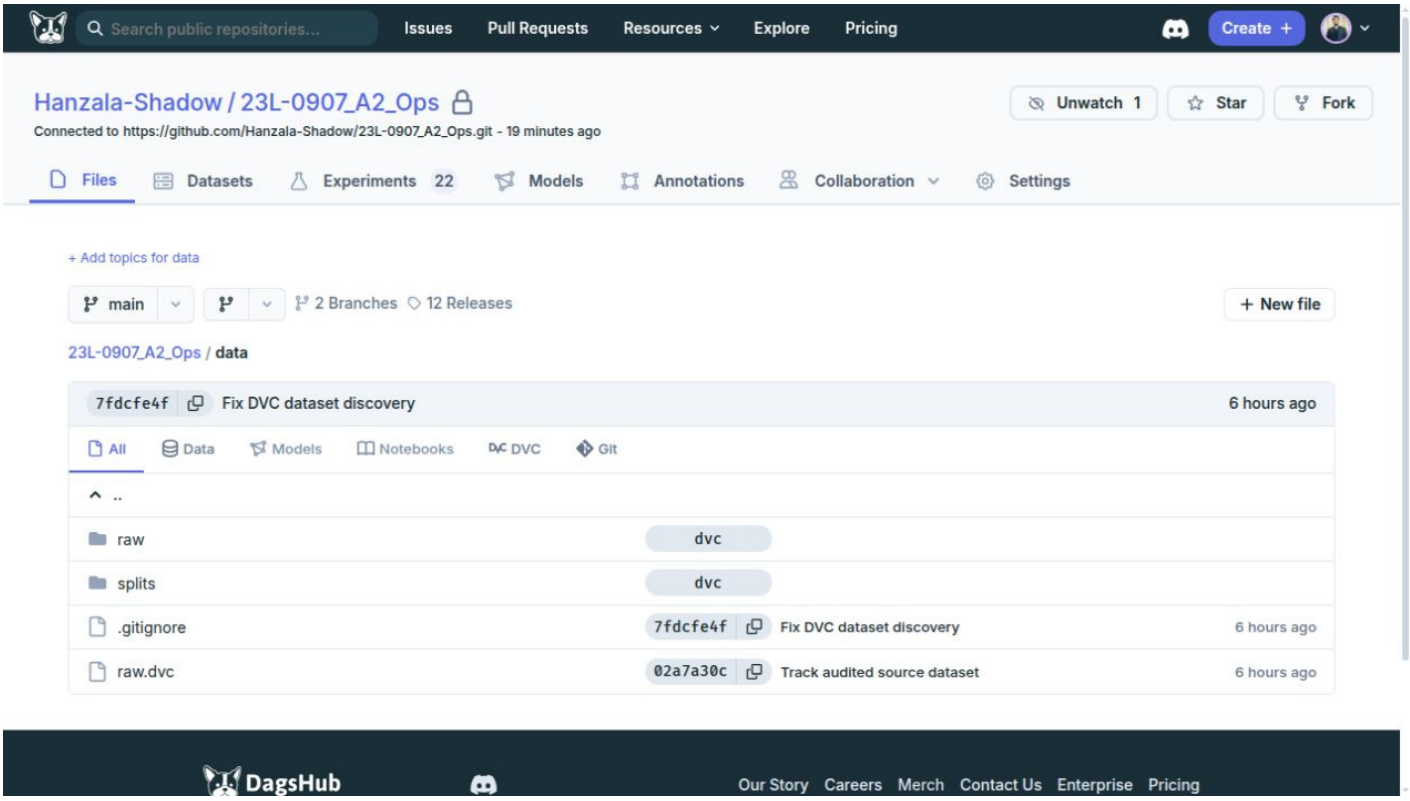


Figure 19. Private DagsHub project connected to GitHub with versioned data pointers.

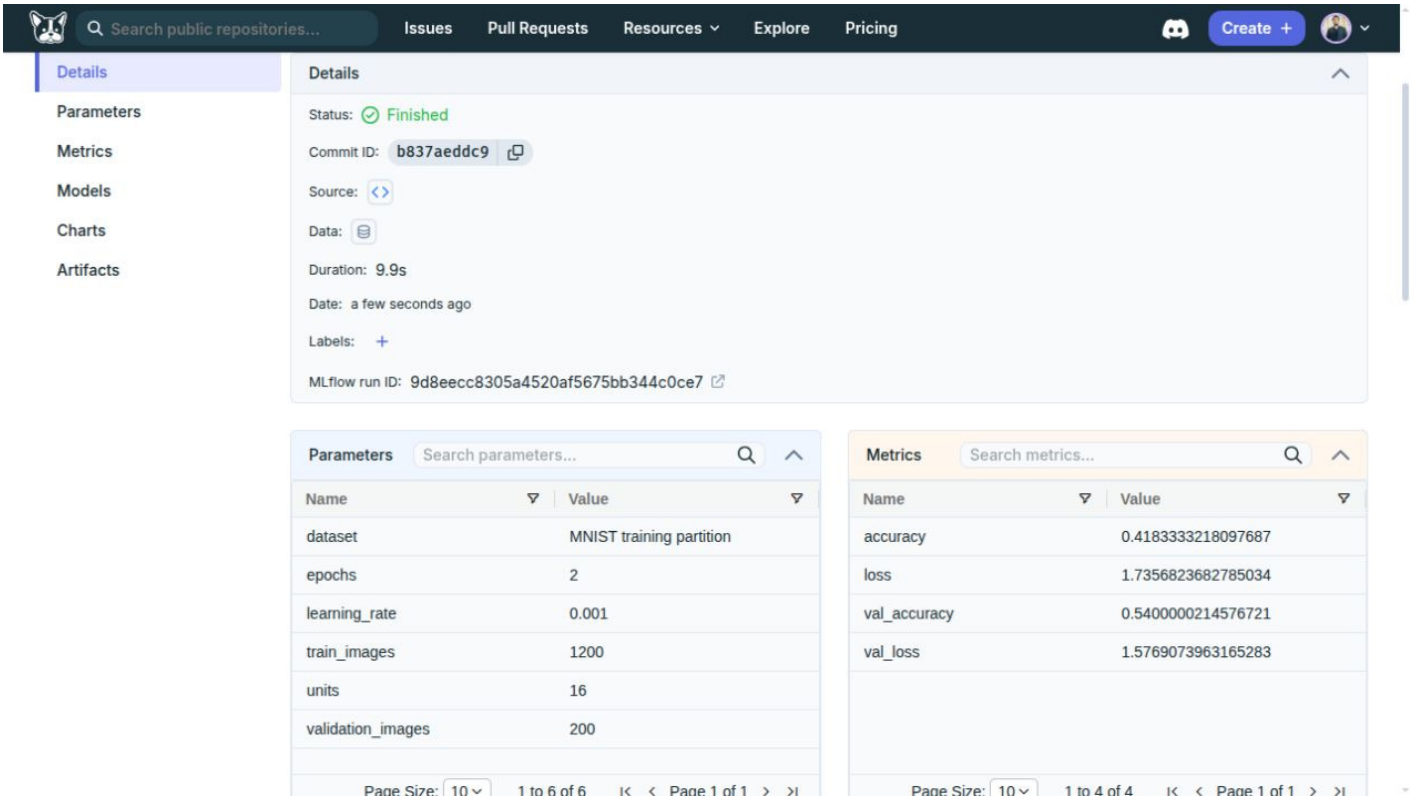


Figure 20. DagsHub toy run, separate from VisionGuard training experiments.

A first remote attempt accidentally retained the local tracking address: MLflow set_tracking_uri updates the process environment. The correction captures the cloud address before selecting local

storage. The local diagnostic run is labeled, and only the corrected remote run is claimed as DagsHub evidence.

3.3 Manual logging and TensorFlow autologging

The CNN uses a Keras callback to log epoch loss, accuracy, AUC, precision, recall and learning rate. It explicitly adds seed, source commit, DVC checksum, split artifacts and selected checkpoint. The two MobileNet stages use TensorFlow autologging for fit parameters, optimizer data, epoch metrics and TensorBoard evidence. Automatic final-model upload is disabled; the best validation checkpoint is logged explicitly. Autologging does not replace project-specific provenance, layer-boundary checks, cost-based threshold choice or exact validation/test metrics. Tuner trials use explicit logging. (Figures 21, 22.)

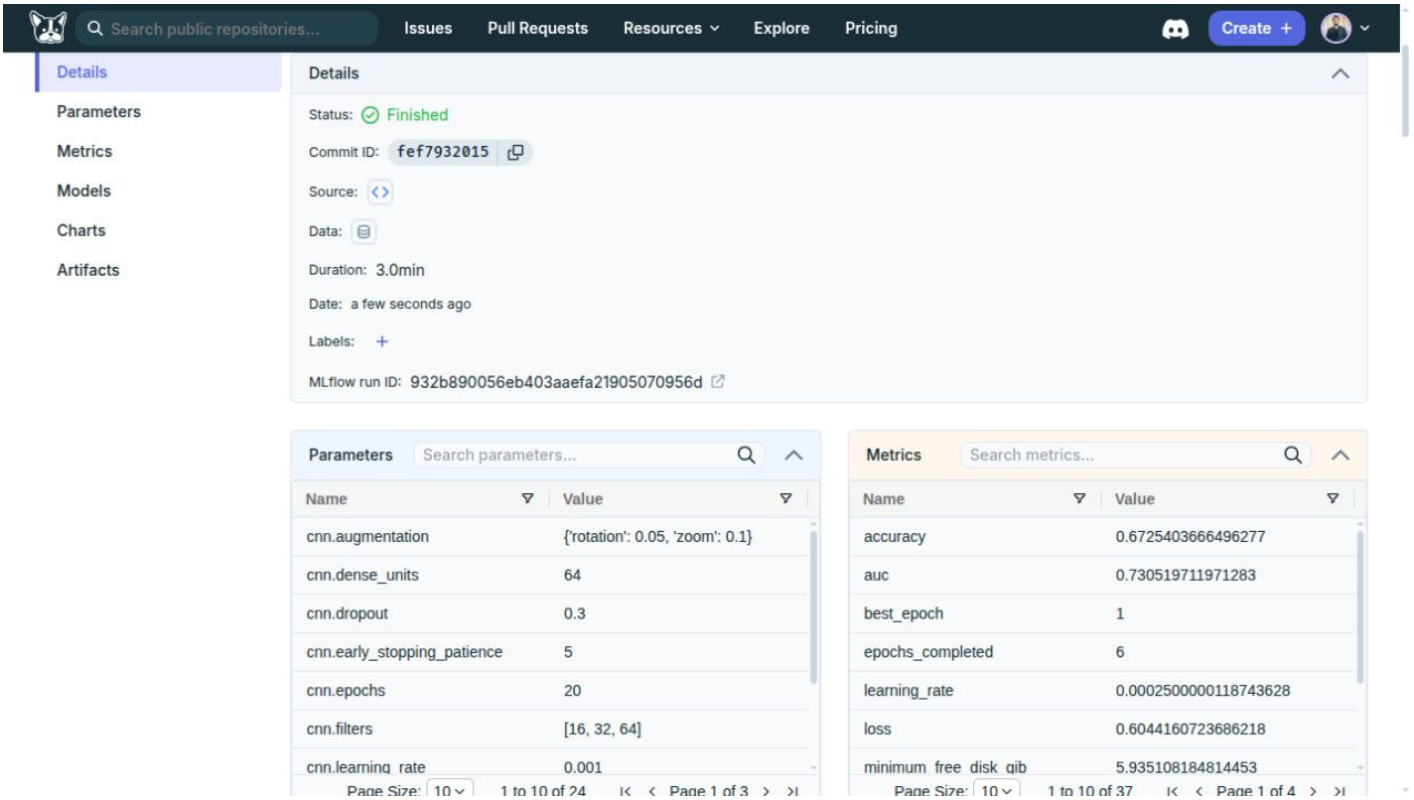


Figure 21. CNN run evidence: manually logged metrics and project context.

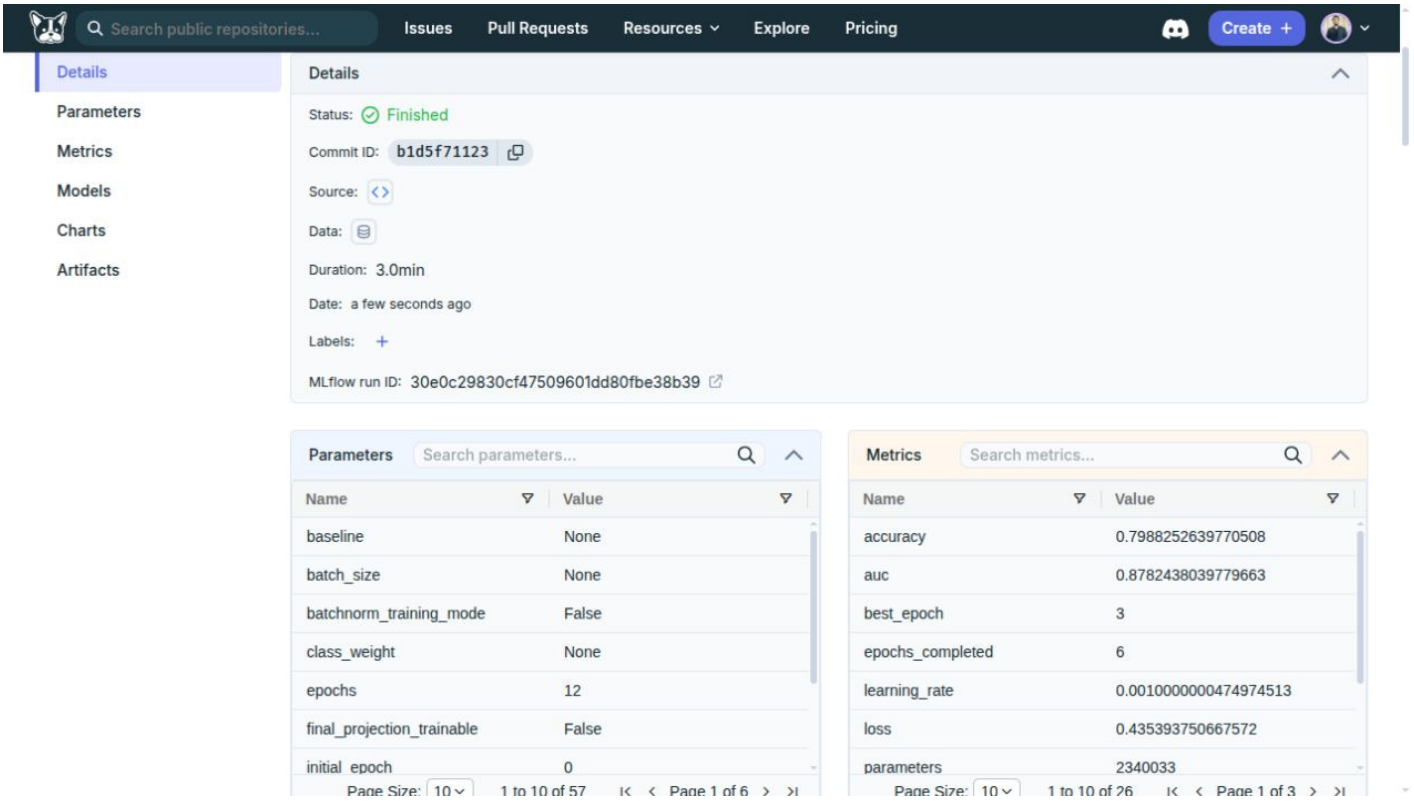


Figure 22. MobileNet frozen-stage evidence with framework autologging and explicit provenance.

3.4 Required artifacts

Training runs include the native Keras checkpoint, MLflow model, configuration, package list, split lists, layer summary, curves and validation outputs. The final evaluation run stores both models' test confusion matrices, ROC curves and saved predictions, and final test metrics are attached to the original training runs. Keeping evaluation separate preserves the original training provenance. (Figure 23.)

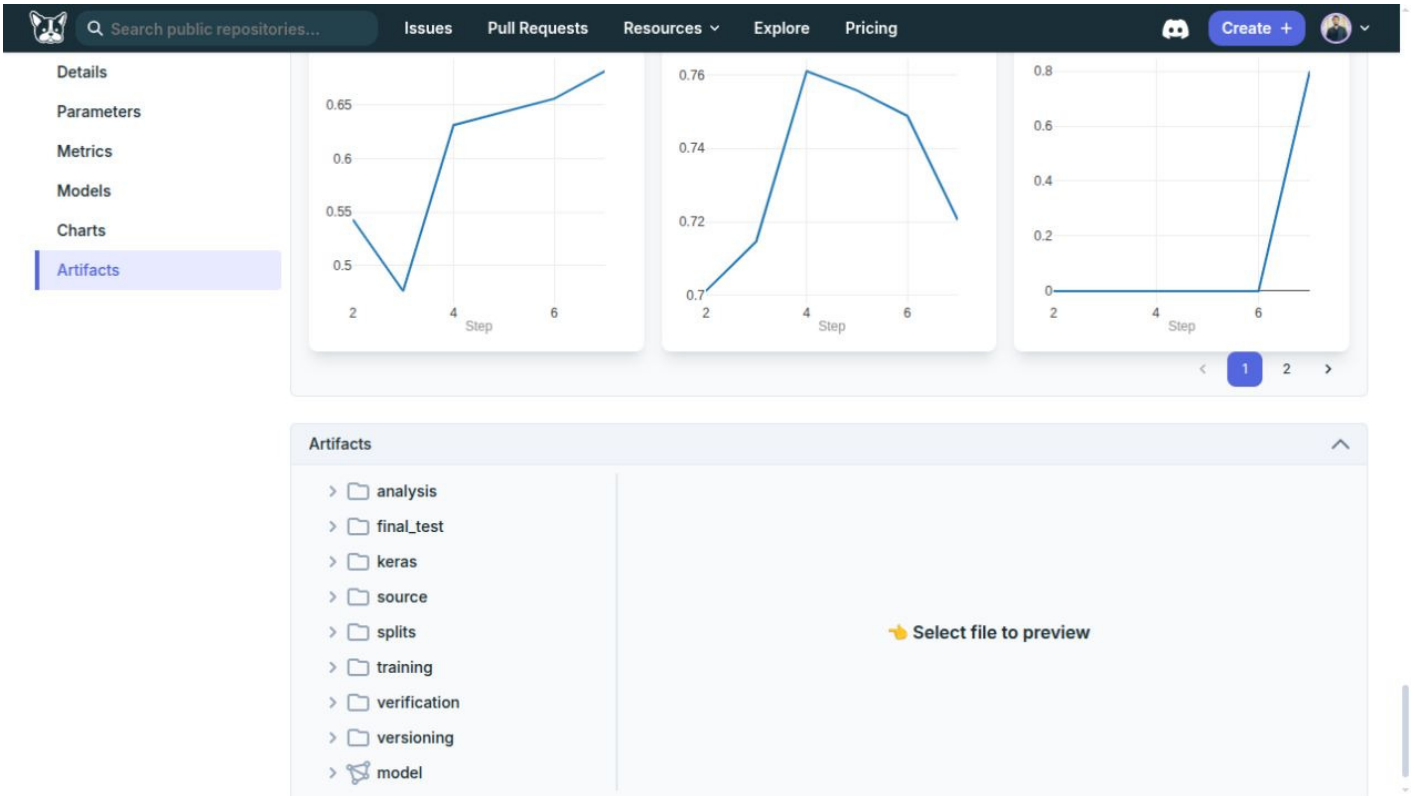


Figure 23. MLflow artifact browser showing saved model and reporting files.

3.5 Meaningful experiment comparison

Candidate	Validation loss	Accuracy at 0.5	ROC-AUC
cnn_baseline	0.70121	0.4452	0.5151
mobilenetv2_frozen	0.49845	0.7808	0.8418
mobilenetv2_finetuned	0.51045	0.7740	0.8427
tuner_0	0.51384	0.7671	0.8439
tuner_1	0.50332	0.7671	0.8431
tuner_2	0.50416	0.7671	0.8429
tuner_3	0.54409	0.7603	0.8496
tuner_4	0.51258	0.7466	0.8397
tuner_5	0.51104	0.7534	0.8389
tuner_6	0.49604	0.7603	0.8498
tuner_7	0.48514	0.7877	0.8663

The table compares the CNN, frozen MobileNet, last-block fine-tune and eight controlled tuner combinations. Minimum validation loss selects the model before cost-based threshold selection. The

frozen network outperformed the first fine-tune; tuned dropout improved the best candidate. Final test results are reported as a check on the locked choice, not as another selection round. (Figure 24.)

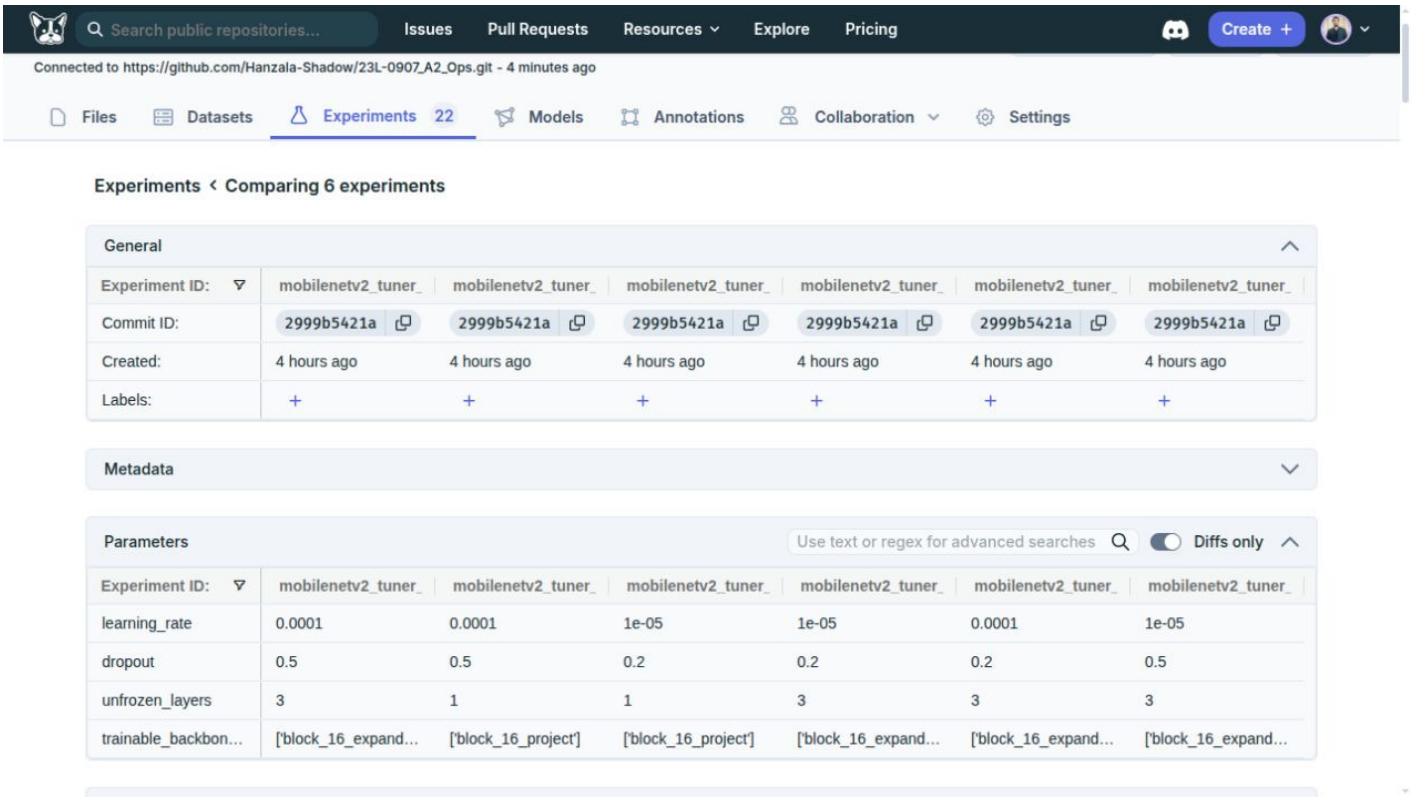


Figure 24. Six selected DagsHub tuning runs with meaningful parameter differences; all eight are tabulated above.

3.6 Registry and alias-based inference

Registry 23L-0907_A2_Ops contains version 1 (frozen MobileNet) and version 2 (tuned MobileNet). The champion alias points to version 2. scripts/predict_registry.py independently loads models:/23L-0907_A2_Ops@champion, applies the shared preprocessing, and classifies a validation image. Its saved probability 0.9492709637 and threshold 0.6196606457 reproduce the selected bundle. No test image is used for that demonstration. (Figures 25, 26.)

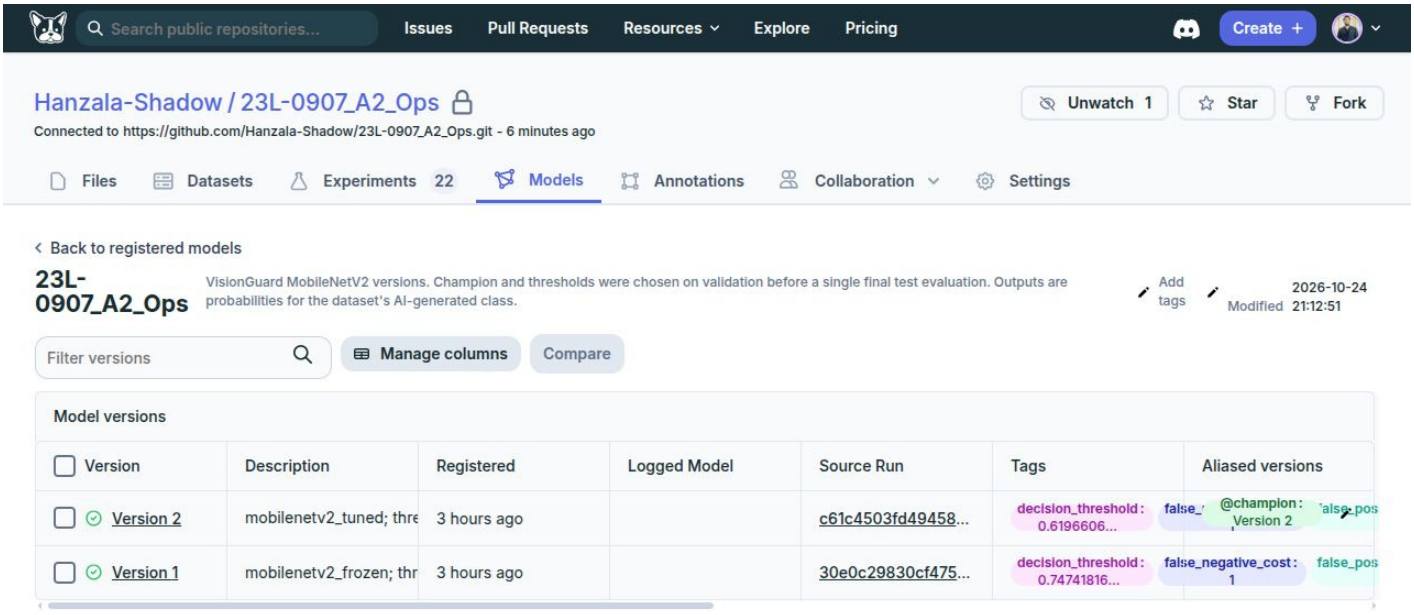


Figure 25. Two registered versions, with champion assigned to version 2.

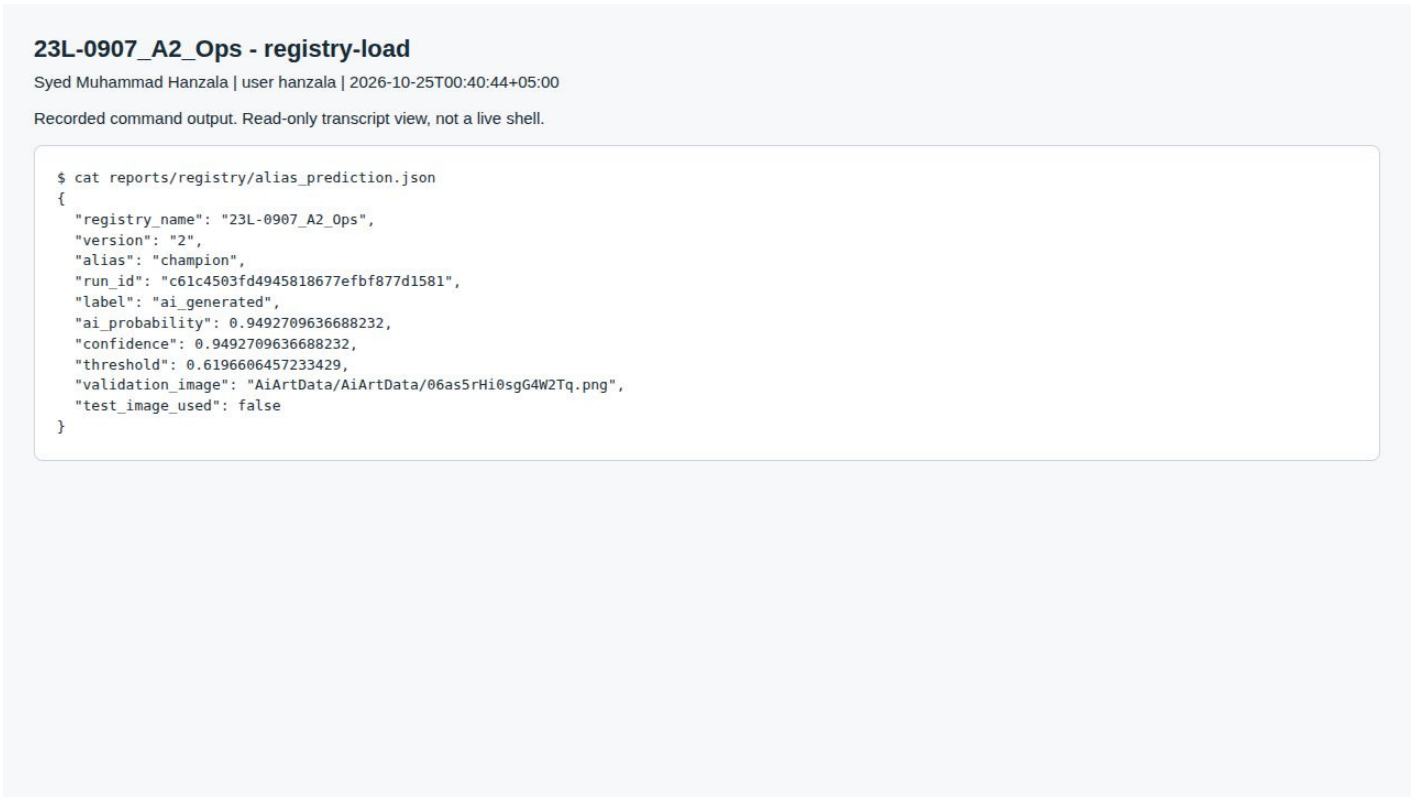


Figure 26. Recorded separate alias-load prediction and model identity.

Task 4 - Flask application

4.1 Inference lifetime and matching preprocessing

visionguard/serving.py loads the native Keras champion once when the worker starts, validates its hash/policy and warms it with synthetic pixels. The native implementation remains available in the retained image and read-only bundle. Current Flask serving uses one RemoteModelService per worker and calls the cloud export; the Vercel process loads its interpreter once per warm instance. A cold start or another instance loads its own copy. DagsHub is not queried on requests.

Loading weights per request would waste CPU, increase latency and memory churn, and risk selecting an inconsistent version. Both native and cloud paths use RGB white letterboxing and normalization inside the model. The float32 export matched all 146 validation decisions; test was not used for conversion. The two original known validation-image checks are recorded in reports/application/verification.json.

4.2 Routes and input validation

GET / displays upload; POST /predict returns a saved result; GET /result/<id> reads it; GET /history paginates database records; GET /health checks readiness. The validator checks the form token, one file, sanitized basename, extension, decoded format, file bytes, decoded pixels and still-image content. The remote contract allows JPEG, PNG and WebP up to 4 MiB and 4 million pixels. Images stay in memory and are discarded; only metadata and results are retained. (Figures 27, 28, 29.)

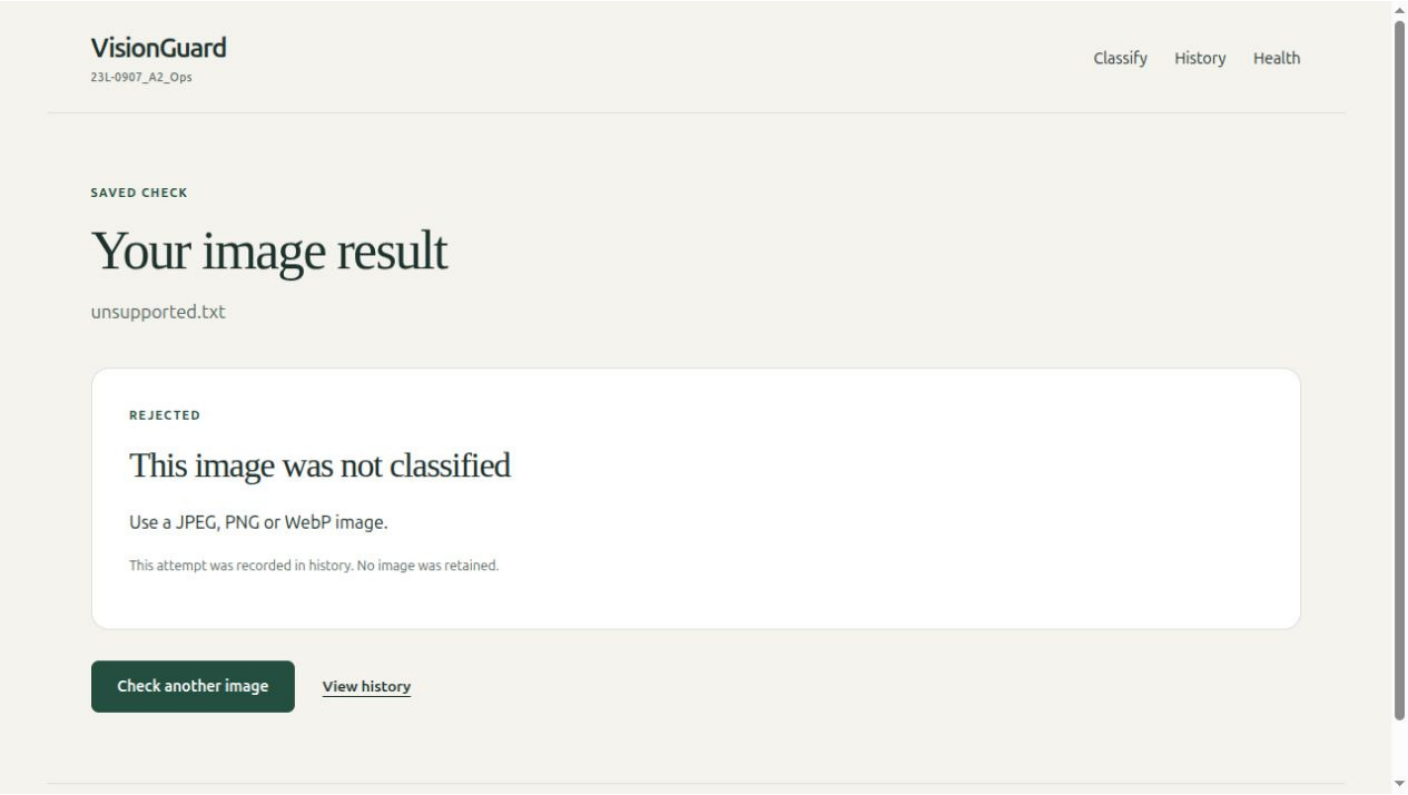


Figure 27. Wrong file type: friendly saved rejection (HTTP 415).

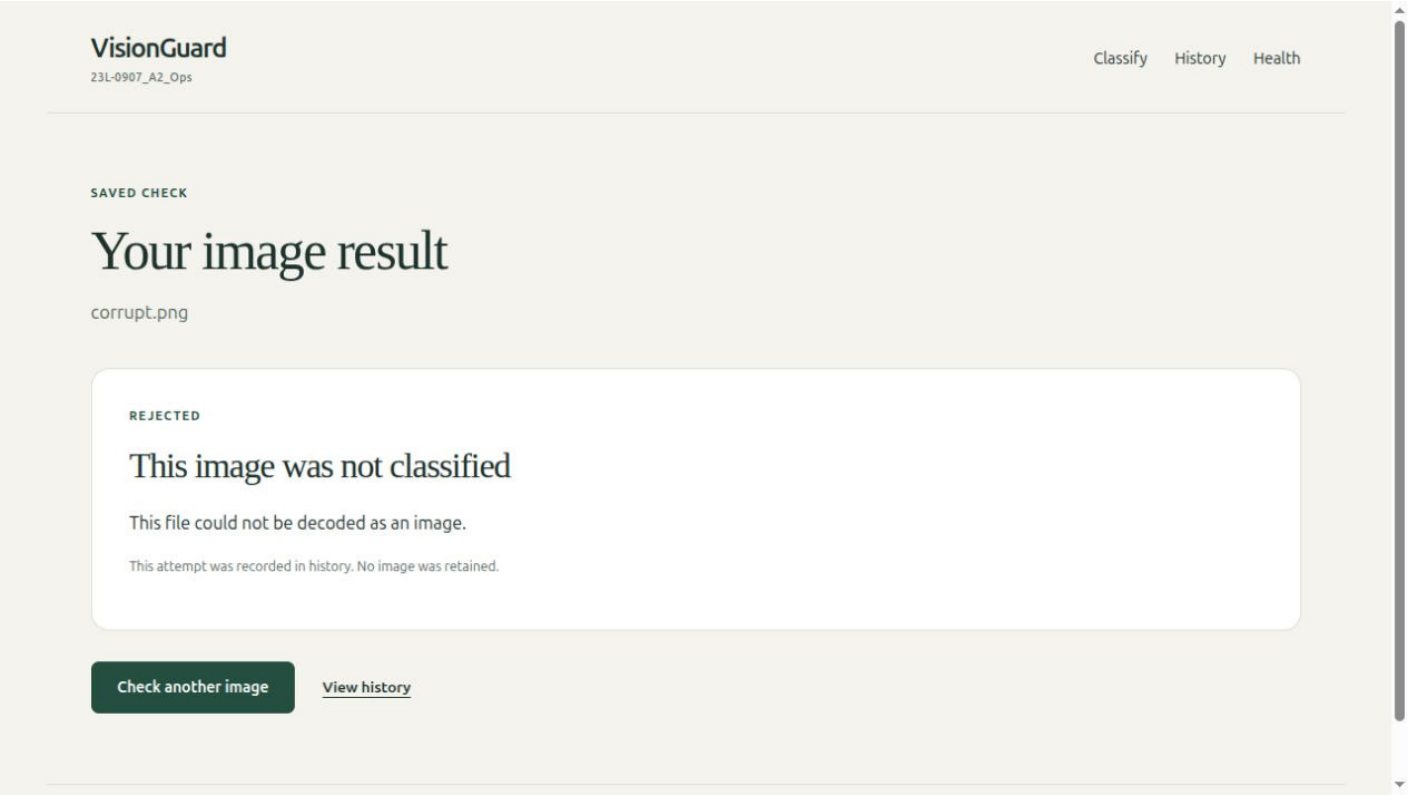


Figure 28. PNG-like bytes that cannot decode: friendly saved rejection (HTTP 400).

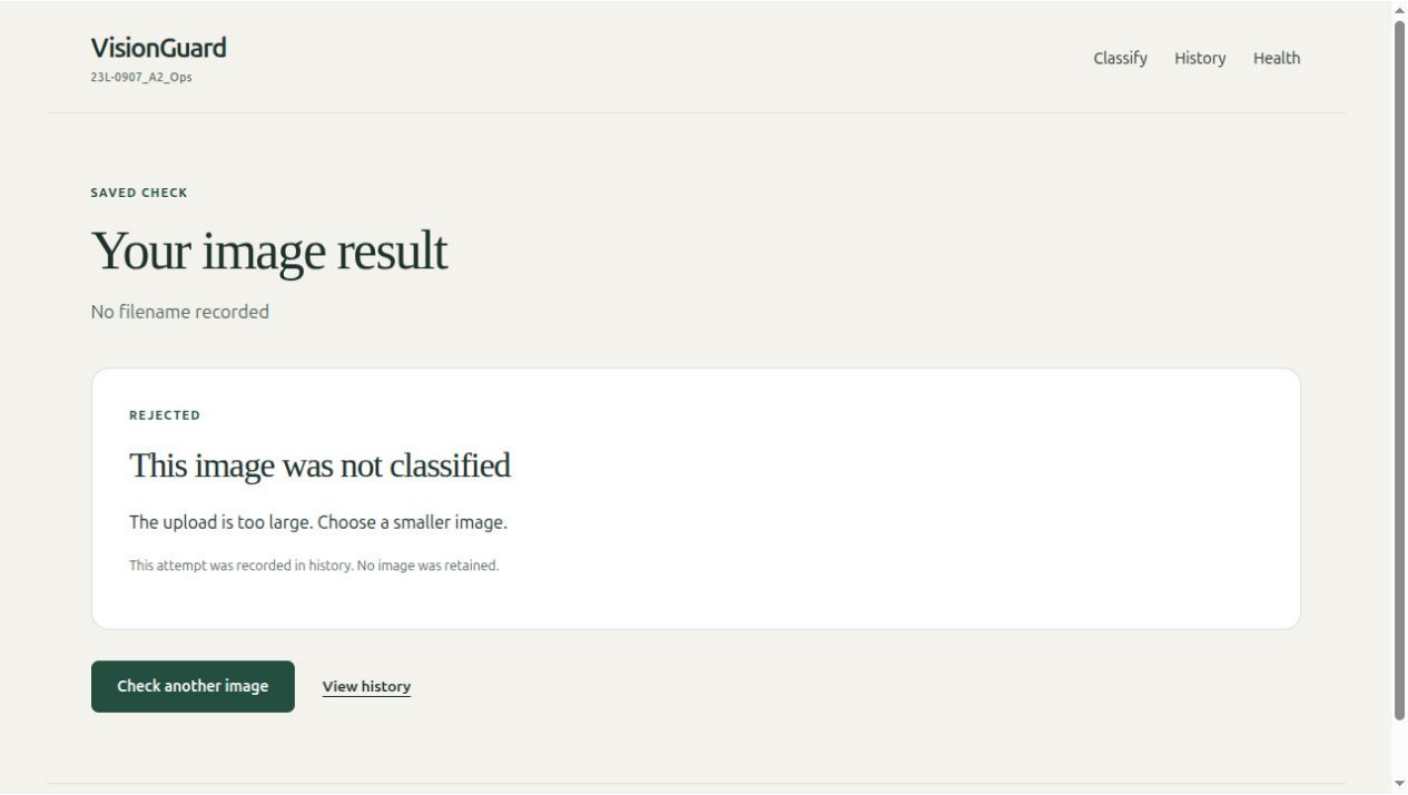


Figure 29. Oversized file: friendly saved rejection (HTTP 413).

4.3 Upload, result and history interface

The HTML/CSS interface offers an upload preview, verdict and confidence, a visible model caveat, and a history menu that reads PostgreSQL. The upload page explains cloud transmission and data retention. The API key is never exposed to browser JavaScript. Unknown result IDs return a friendly 404. (Figures 30, 31.)

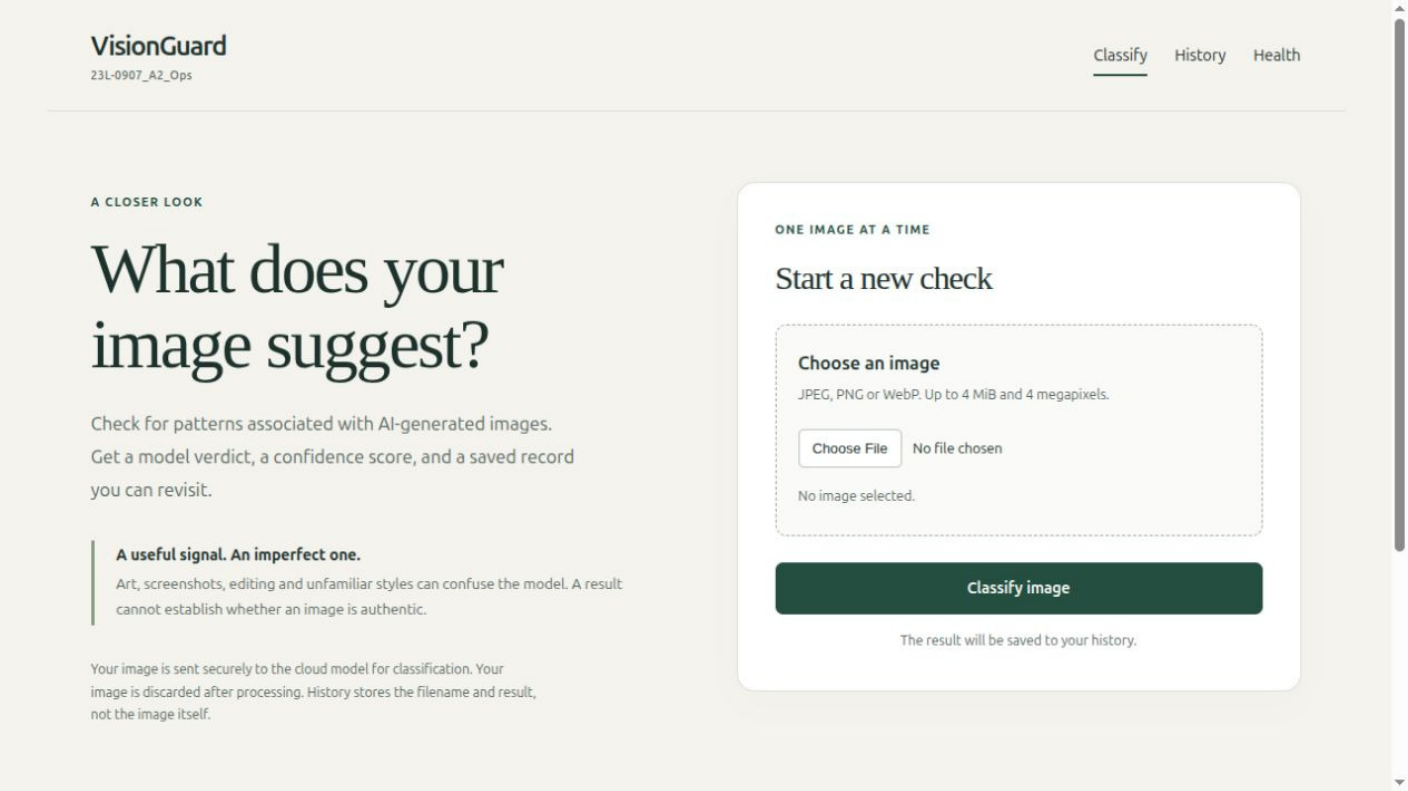


Figure 30. Local upload interface with limits and cloud-transmission disclosure.

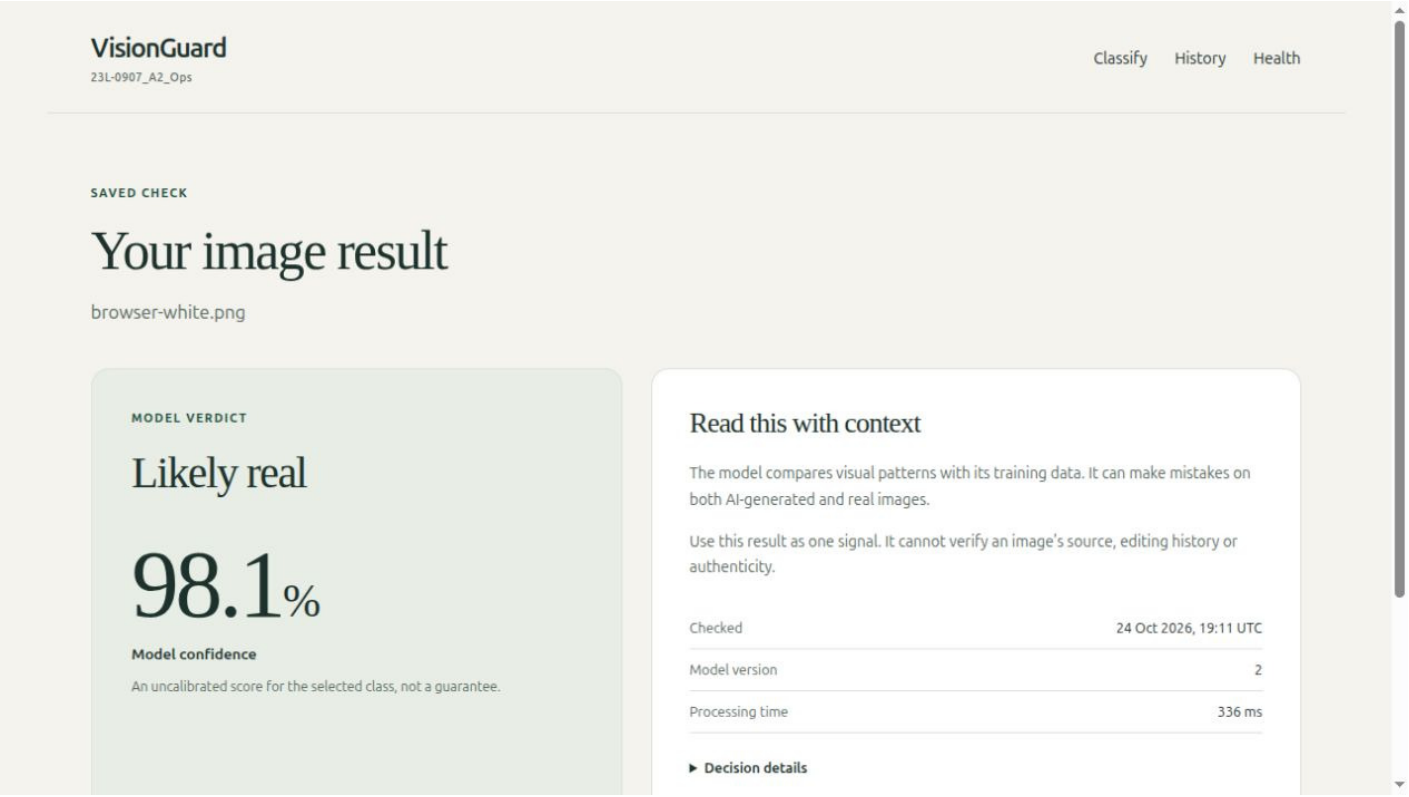


Figure 31. Saved cloud-backed result showing verdict, confidence, model version and caveat.

4.4 Environment settings, debug and server

Model/policy paths, saved threshold, database URL, session secret, disk reserve and remote settings come from environment variables. A supplied threshold must match the saved policy. Debug is disabled to avoid interactive tracebacks and development behavior. The app was checked under Flask's server during development; Compose uses Gunicorn with one worker and two threads. Health reports a remote backend without claiming local weights are loaded. (Figure 32.)

Pretty-print ☑


```
{
  "database": {
    "backend": "postgresql",
    "status": "ok"
  },
  "debug": false,
  "disk": {
    "free_gib": 2.404,
    "reserve_gib": 2
  },
  "model": {
    "backend": "vercel",
    "error_code": null,
    "loaded": false,
    "name": "23L-0907_A2_Ops",
    "status": "ok",
    "threshold": 0.619660645723343,
    "version": "2"
  },
  "status": "ok"
}
```


Figure 32. Current health JSON: PostgreSQL, Vercel model, saved threshold, debug off and disk reserve.

Task 5 - PostgreSQL database trail

The predictions table records UUID, UTC time, sanitized filename, image SHA-256/bytes/dimensions, status, error code/message, label, AI probability, confidence, threshold, processing time, model name, version, run ID, artifact hash and preprocessing policy. SQLAlchemy uses short transactions and parameterized queries. A successful prediction is not shown as saved if the database write fails. Invalid uploads and remote failures are recorded with safe messages; failed attempts have no verdict.

```
SELECT created_at, status, label, confidence, model_version
FROM predictions ORDER BY created_at DESC LIMIT 10;
```

The history menu renders these rows rather than a browser-only list. UTC is labeled in the UI. A named Docker volume preserves history across container recreation. Saved snapshots compared all original row fields, not just counts. All 11 original records survived Docker and cloud integration; the eight explicitly named integration/browser checks brought the count to 19. This final audit does not delete those records or rewrite old model hashes. (Figure 33.)

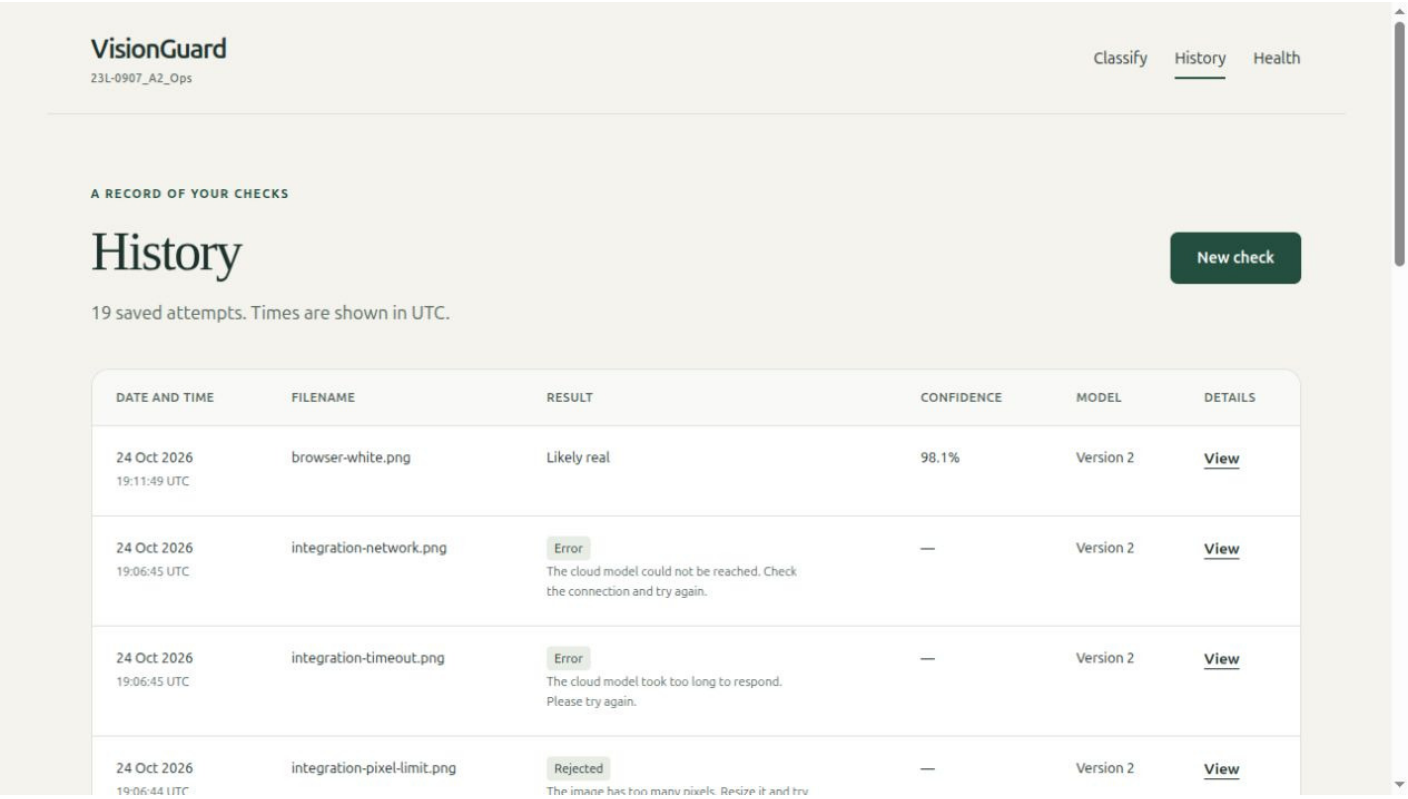


Figure 33. PostgreSQL-backed history, including successful checks and safely recorded failures.

Task 6 - Docker and continuous integration

6.1 Docker warm-up

A disposable official postgres:17-alpine container demonstrated run, loopback port mapping, a new named volume, logs, shell access, listing, stop, start and restart. A temporary image tag was created and removed, then only the lab container and lab volume were deleted. The original image was retained because the real database uses it. Production containers and their volume were not touched. (Figure 34.)

Term	Meaning
Image	An immutable template of filesystem content and runtime defaults.
Container	A running or stopped instance with its own process and writable state.
Layer	A cached filesystem change produced by an image build instruction.
Volume	Docker-managed storage that outlives a container.
Port mapping	Forwarding a selected host address/port to a container's port.

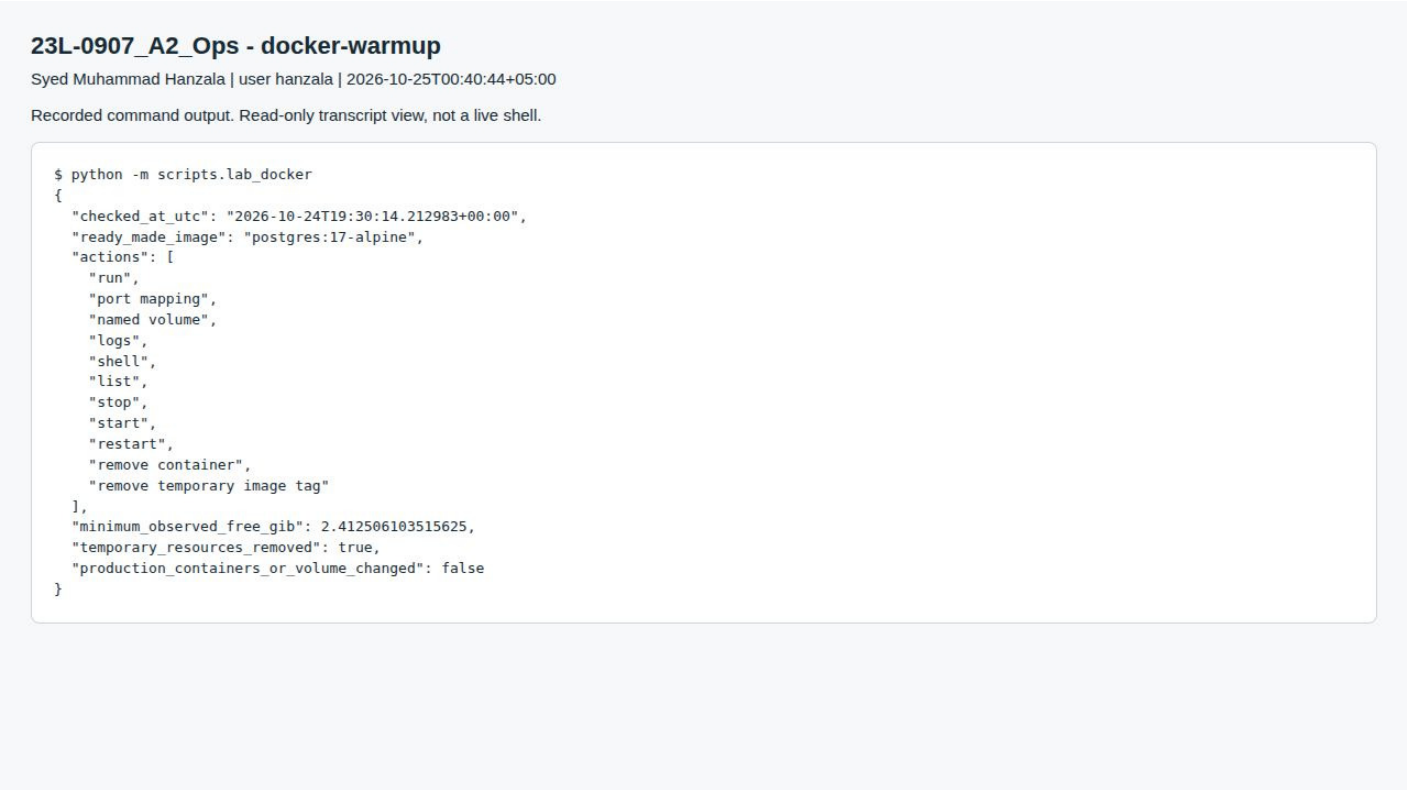


Figure 34. Recorded ready-made image lab commands and cleanup receipt.

6.2 Dockerfiles and secret handling

Dockerfile builds the CPU-only native server; Dockerfile.client builds the current lightweight remote client. Both start from pinned Python slim images, install pinned dependencies before copying source, run as UID 10001 and include an HTTP health probe. Dependency-first ordering preserves the large

install layer when only source changes. Deny-by-default ignore files allow only serving inputs. Dataset images, weights, Git metadata, .env files and training tools do not enter image layers.

The native image deliberately uses tensorflow-cpu and removes unused development headers. The client needs Flask, PostgreSQL, Pillow and Requests only. It retains the saved policy mount but does not install a local model runtime. The actual image is imported from a verified private CI artifact to avoid a local rebuild. The downloaded archive was removed after import.

6.3 Two-service Compose stack

compose.yaml defines web and db; compose.cloud.yaml overrides web for remote inference. The setup script preserves database and session secrets and sets COMPOSE_FILE so normal Compose commands select both files. web uses db:5432 on the private Compose network, waits for the database health check, and publishes only 127.0.0.1:5000. PostgreSQL's administration mapping is loopback-only 127.0.0.1:5433; there is no public bind. Passwords come from ignored .env and placeholders are provided in .env.example. The external named history volume is reused, and the champion directory is mounted read-only. The root filesystem is read-only with a bounded temporary memory mount. (Figure 35.)



Figure 35. Recorded healthy services, UID, volume, read-only mount and loopback-only port bindings.

6.4 Size and layer analysis

The same docker image ls measurement reports 1.72 GB for the retained native TensorFlow image and 395 MB for the remote-client image, a reduction of about 77%. This optimization removes TensorFlow and numerical model dependencies from the local web image after moving inference to Vercel. It trades offline inference for network dependency; native serving remains available.

Registry/archive compressed byte sizes are different measurements and are not mixed with these values.

docker history identifies dependency installation as the largest layer in both images, with the native TensorFlow install layer at 1.22 GB versus the client's 63.9 MB install layer. Slim base, no pip cache, no compiler and an allowlisted build context reduce size further. The exact history output is in reports/labs/docker/verification.json.

6.5 Operations, failure diagnosis and recovery

The isolated lab also ran the native web image against its disposable database. Omitting the champion mount made the Gunicorn worker exit. Logs showed FileNotFoundError for policy.json, which identified a missing serving bundle rather than a database problem. Adding the existing bundle as a read-only mount restored HTTP 200 with loaded=true and healthy PostgreSQL. Shell, logs, stop, start and restart were exercised for both services, and all temporary resources were removed. (Figure 36.)

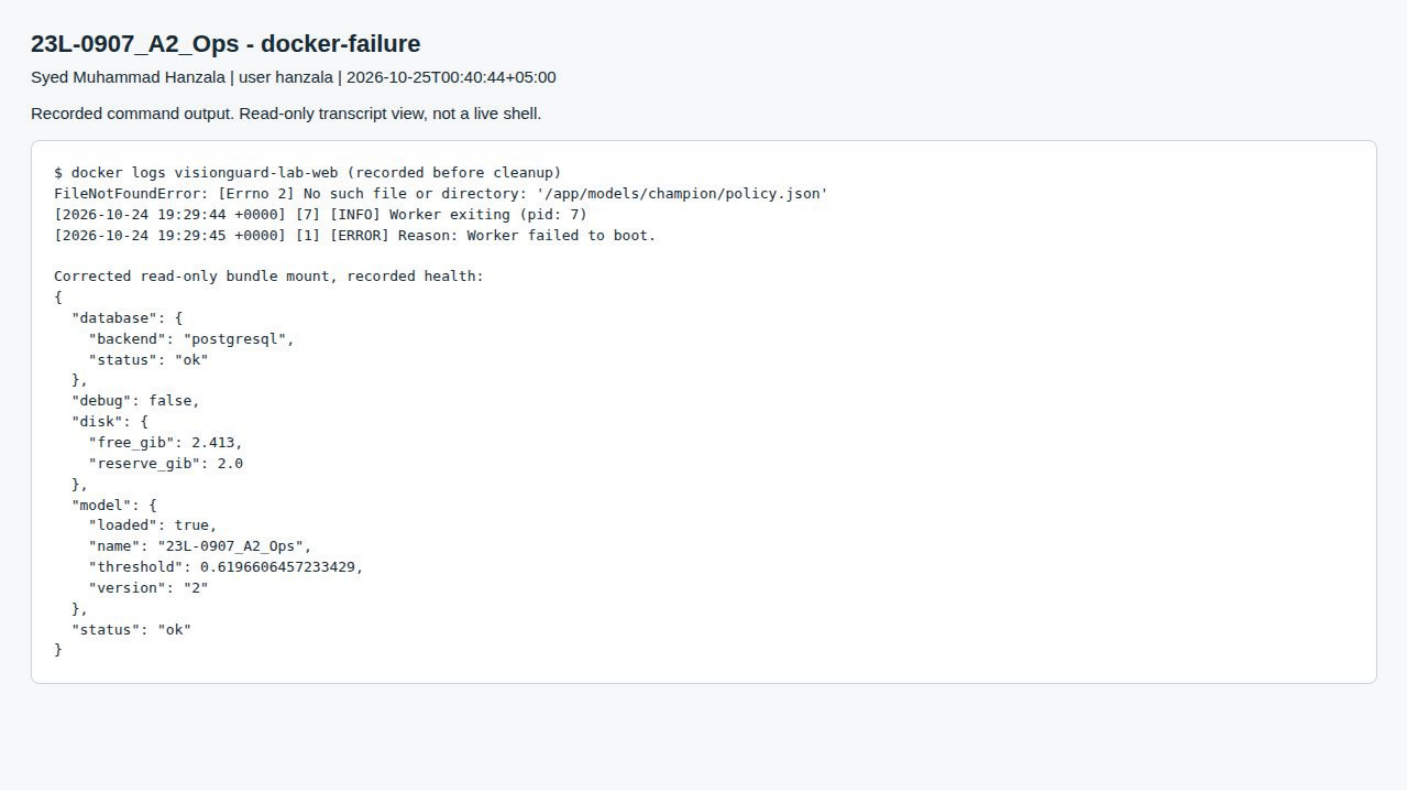


Figure 36. Actual missing-policy worker log and the corrected healthy native model response.

Earlier production checks also recreated PostgreSQL and confirmed the original rows after recovery. A temporary outage returned 503 and then recovered without restarting the Gunicorn worker. These receipts are retained under reports/container and reports/application.

6.6 GitHub Actions red/green lab and project CI

The separate authorized private CI lab has a Python 3.11/3.12 matrix. Its initial arithmetic test expects 2+2=5 and fails on both versions; the next commit corrects the expectation to 4 and passes. The main repository workflow installs pinned dependencies, checks repository content, runs Ruff and synthetic tests, builds the native image, starts an isolated Compose stack, validates five uploads, recreates

containers and checks persistence. It also builds the lightweight client and checks that TensorFlow, MLflow and DVC are absent. The README contains the CI badge. (Figures 37, 38, 39.)

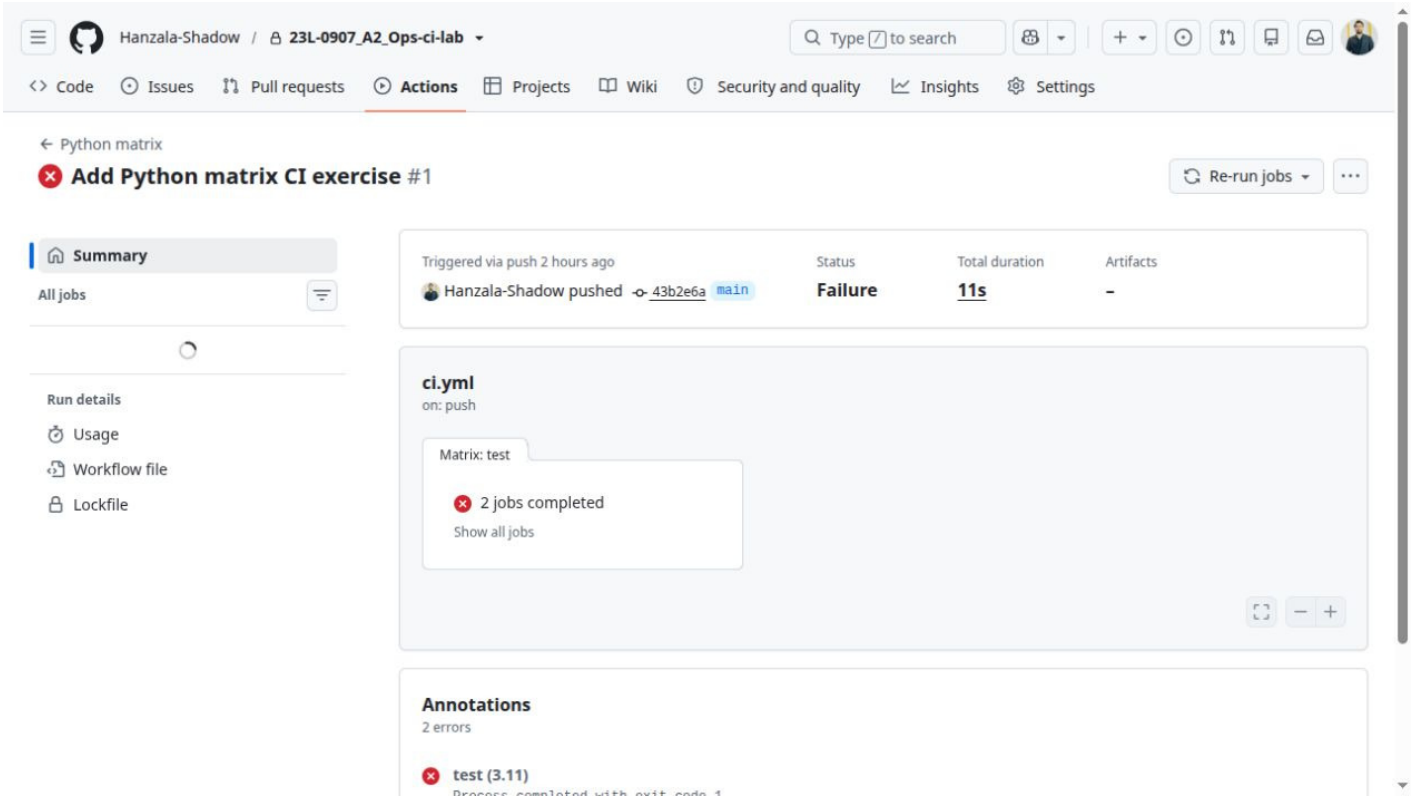


Figure 37. The toy matrix fails on Python 3.11 and 3.12 as intended.

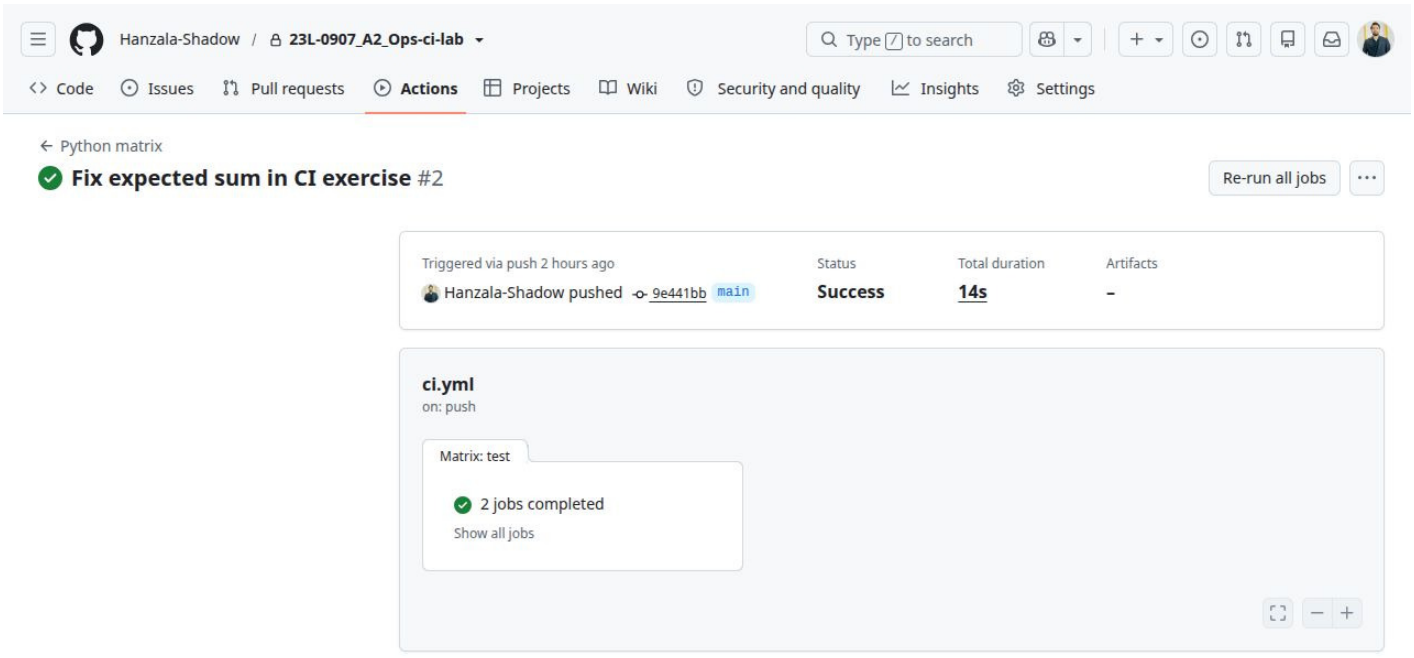


Figure 38. The corrected toy test passes on both Python versions.

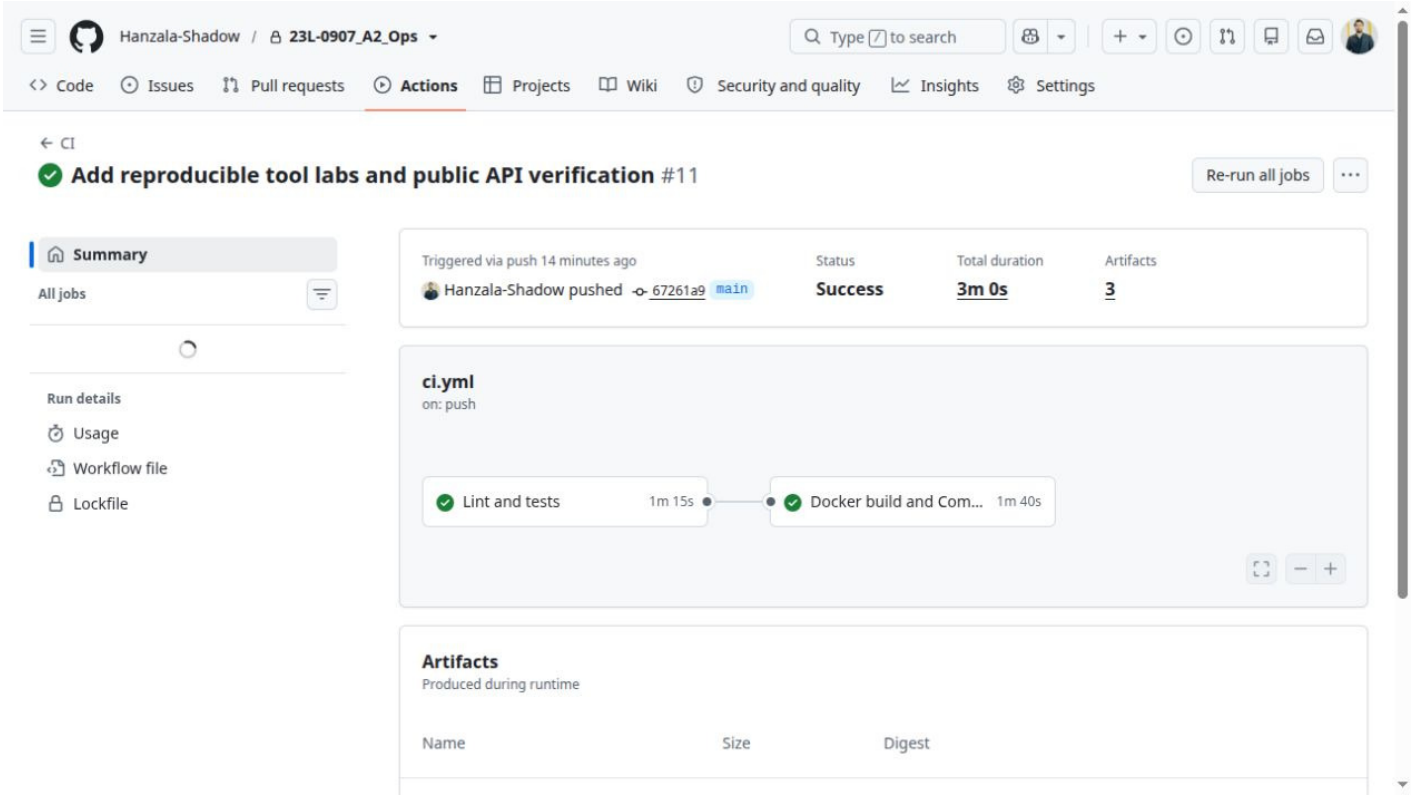


Figure 39. Main project CI passes lint, tests, Docker builds and Compose checks.

CI's generated database/session settings stay in ignored runner files. Actions secrets belong in GitHub repository or protected-environment secret storage and are referenced by name, never embedded in YAML. The separately authorized API-only secret is used solely by the manually triggered public API check. Its reports omit request headers and bodies and its temporary secret file is removed. No training account credential is available to that job.

Task 7 - Cloud model API deployment

Deployment design and limits

The model API is on Vercel Hobby, with no billing details or paid add-ons. A float32 LiteRT export avoids the TensorFlow function-size problem while preserving the Keras champion. The runtime measured roughly 179 MiB locally and fits the provider's Python bundle allowance. Request bodies have a provider limit of 4.5 MB, so the app caps files at 4 MiB plus multipart headroom. Cold starts can add delay. Hobby is for personal non-commercial use and quota exhaustion can pause service; this is a free-tier demonstration rather than an availability guarantee.

Deployment steps and rationale

Step	Action and purpose
1	Lock the validation-selected champion; export float32 LiteRT with inference behavior and verify all 146 validation decisions. Do not use test.
2	DVC-version the export; create a model-only private release archive. Pin archive and file SHA-256 values in cloud/model-manifest.json.
3	Create the Vercel Hobby Python project in fra1. Keep the course project identity; use a DNS-safe deployment name.
4	Use pyproject.toml to point at cloud.wsgi:app with pinned lightweight dependencies. vercel.json fetches and verifies the model at build time.
5	Supply a fresh short-lived model-only download URL and a new API key as encrypted production settings. Keep all account tokens and local history off the provider.
6	Deploy the reviewed source after CI. Keep automatic Git deployment disabled. Confirm READY, public health and authenticated prediction.
7	Run the versioned Postman collection from GitHub's hosted runner, a network separate from the laptop, and retain the credential-free results.

The earlier Render Free route required card verification. No billing details were added and no Render service was created. Vercel was selected with the user's approval. Initial Vercel SSO protection was explicitly disabled only for this API project; POST /predict still requires its API key. The model download URL is refreshed before a future build because it expires. The serving API uses its bundled model and does not fetch DagsHub or the release on each request. (Figure 40.)

Pretty-print ☑


```
{
  "model": {
    "format": "tflite-float32",
    "name": "23L-0907_A2_Ops",
    "sha256": "58bd282bcf44bc025c8c844410f0a682cbe2c003676932eff9b43fa66a066128",
    "source_sha256": "4445d4ab52c7d00020bec974faa01f23fff9935284e1d437cf163791b343e0bd",
    "threshold": 0.619660645723343,
    "version": "2"
  },
  "status": "ok"
}
```


Figure 40. Public cloud health response identifies the exact export, source model, version and threshold.

Postman test and logged result from another network

Request	HTTP status	Response ms
Health	200	360
Missing key rejected	401	396
Generated red	200	228
Generated blue	200	316
Generated white	200	226
Wrong file type rejected	415	398

Official Postman CLI 1.76.0 ran the local collection on a GitHub-hosted Ubuntu runner: six requests, 13 assertions, zero failures. The three generated images returned consistent scores/verdicts and the pinned champion hash and threshold; missing authorization returned 401 and invalid text returned 415. No Postman account or cloud workspace was used. The key traveled only to the approved encrypted GitHub Actions secret and the intended Vercel prediction endpoint. (Figures 41, 42.)

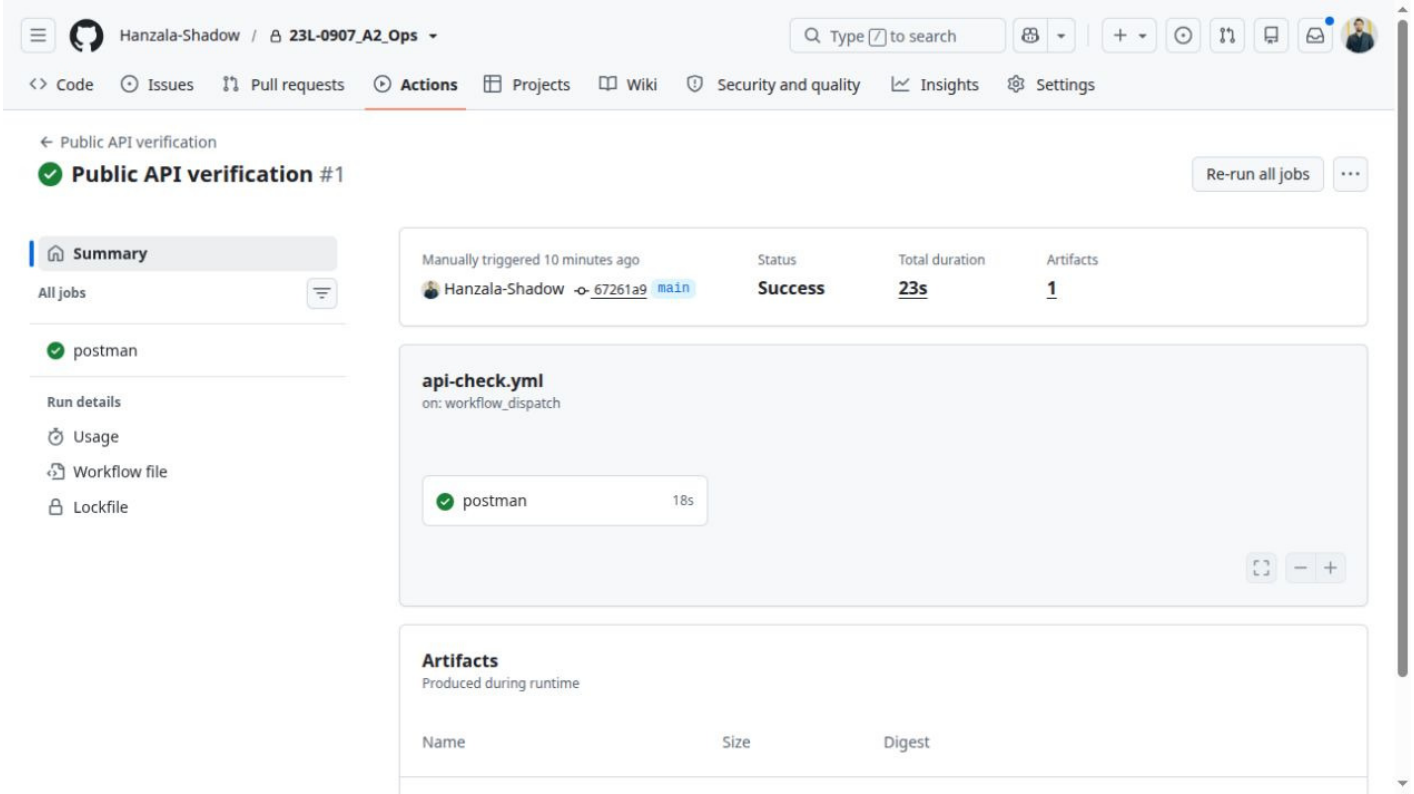


Figure 41. Successful Postman workflow from a separate network; the private artifact contains header/body-free results.

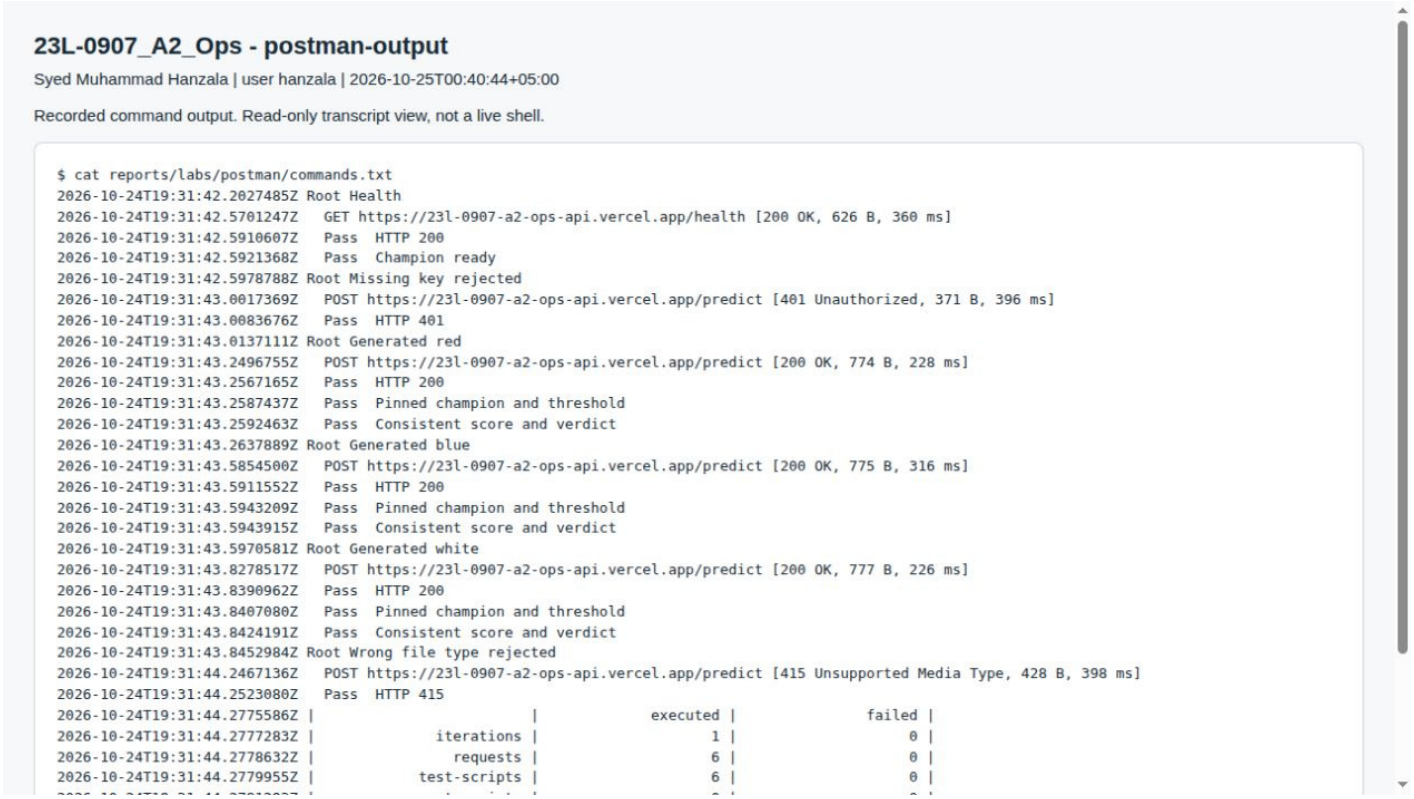


Figure 42. Actual Postman CLI output: six requests and thirteen passing assertions.

Run: https://github.com/Hanzala-Shadow/23L-0907_A2_Ops/actions/runs/40125682249. The importable collection is postman/visionguard.postman_collection.json. Add the API key to a private local Postman environment when testing manually; never paste it into the collection or URL.

Task 8 - Local Flask and cloud API integration

Local validation runs first, then the server sends a generic upload filename over verified HTTPS to the pinned Vercel origin. It reuses connections per thread, uses a 3-second connect timeout and 45-second read timeout, disables redirects and does not automatically retry POSTs. The client bounds JSON responses to 16 KiB and verifies model/version/hashes, saved threshold, finite probability, verdict, confidence and input dimensions before storing a success.

Generated input	AI probability	Verdict	PostgreSQL result
Red	0.0782903	real	saved success
Blue	0.1078499	real	saved success
White	0.0190690	real	saved success

These solid-color images test wiring and parity, not model accuracy. Their cloud results match the local LiteRT reference within 0.0001. The same browser upload/result/history path was exercised with a generated white PNG. No dataset file was sent to the cloud for these checks. (Figures 43, 44, 45.)

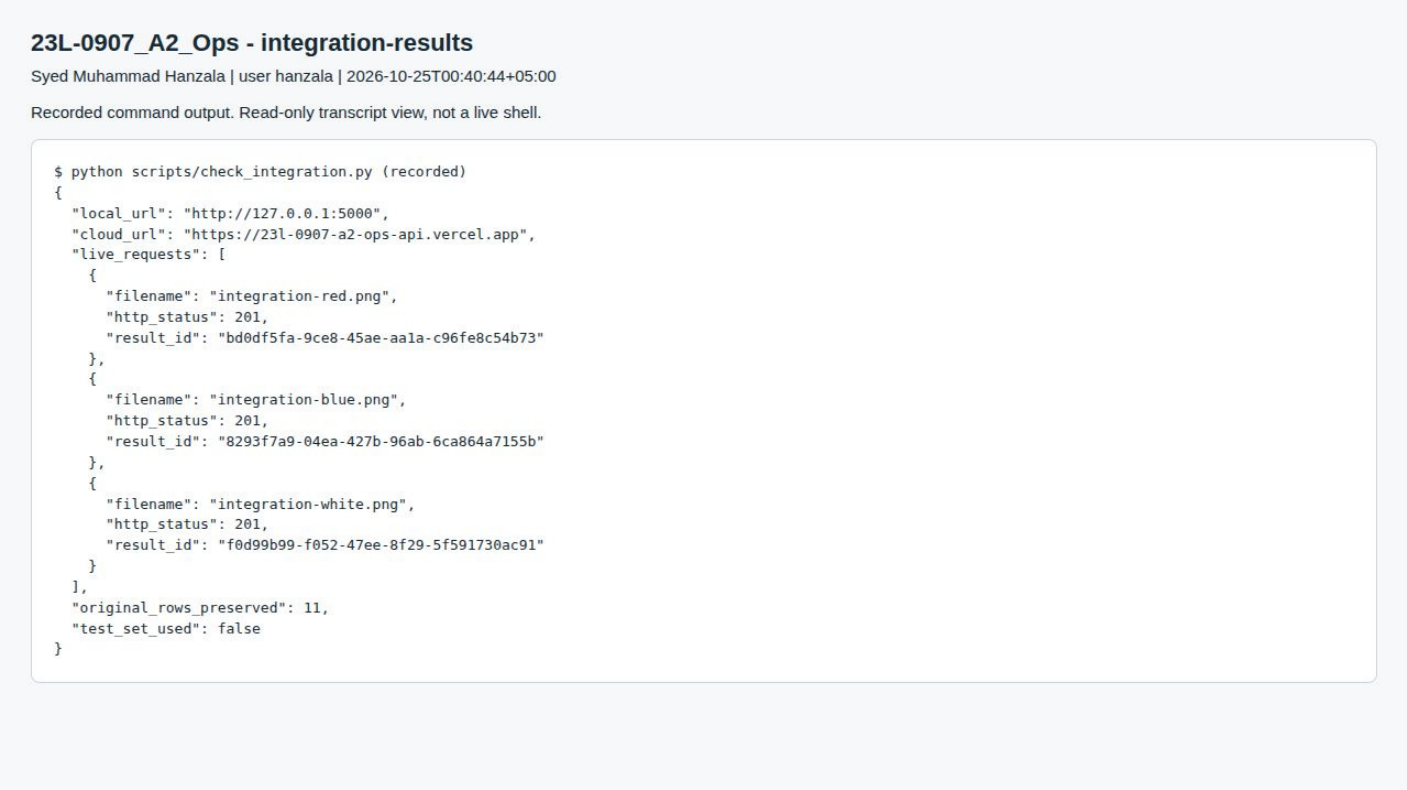


Figure 43. Recorded multiple localhost-to-cloud results and their PostgreSQL identifiers.

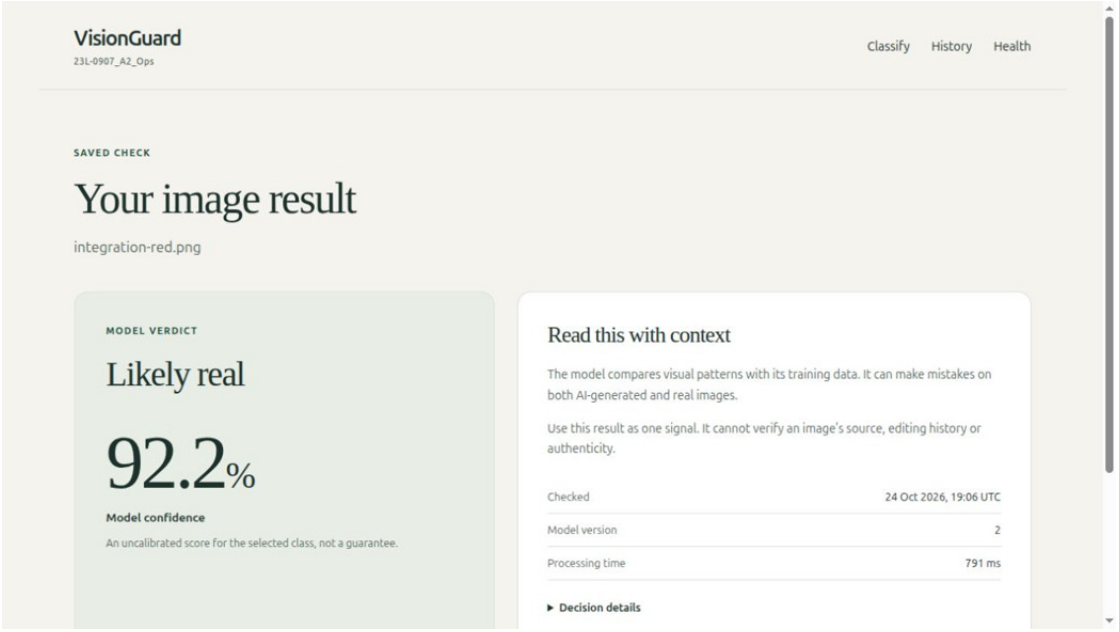


Figure 44. Saved red-input cloud result, read back through the local Flask interface.

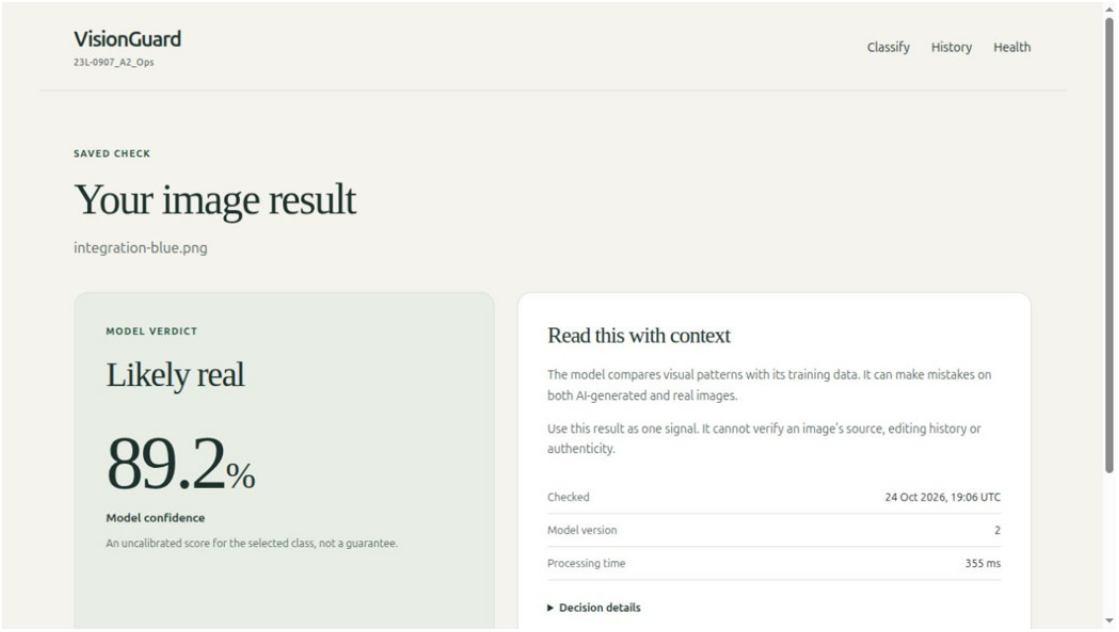


Figure 45. Saved blue-input cloud result, read back through the same local interface.

Failure	HTTP result	Recorded behavior
Connect/read timeout	504	cloud_timeout; no successful verdict
Network/TLS/upstream failure	503	cloud_unavailable; safe retry guidance
Busy/rate-limited model	503	cloud_busy; Retry-After
Wrong API key	503	cloud_authentication; connection needs attention
Malformed or mismatched result	502	cloud_invalid_response; no accepted score

Timeout and network failures were injected into isolated Flask test applications sharing PostgreSQL; the running web container and cloud service were not interrupted. Both failures wrote error rows with null prediction fields, and history remained readable. Health probes have short bounded timeouts and cache readiness. There is no silent local-model fallback. All 11 original rows and the native champion files remained byte-for-byte unchanged. (Figure 46.)

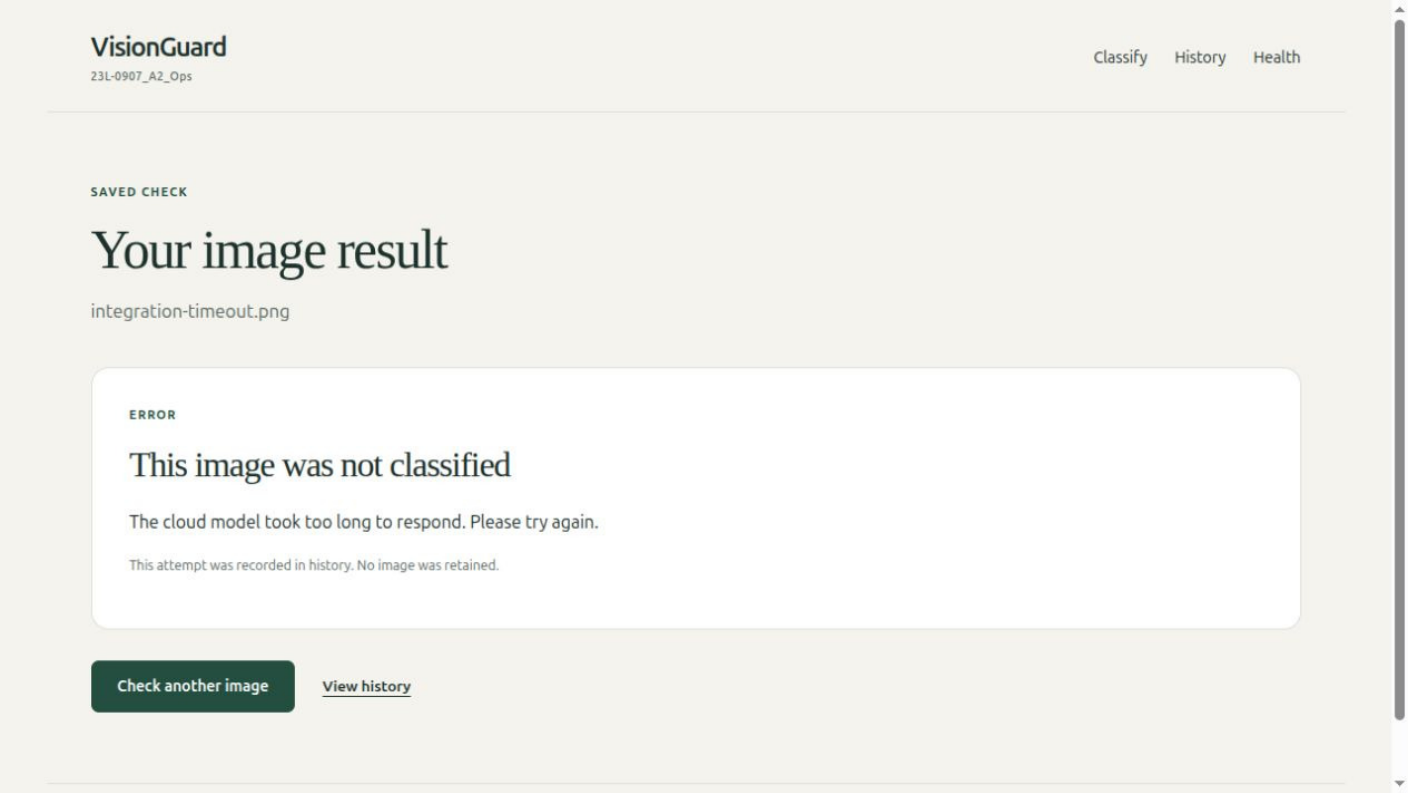


Figure 46. Saved timeout error is readable through the live local app and links back to history.

The production client uses the same local PostgreSQL volume, persistent session secret and saved threshold. Upload guards require 2 GiB plus a 128 MiB safety buffer. Logs rotate, uploaded files are not persisted, and disposable lab resources were removed. The minimum observed free space during the final Docker lab was 2.41 GiB, above the reserve.

Submission checklist and reproducibility

Item	State
Named PDF	23L-0907_VisionGuard.pdf, with name, links, source hash, assistance disclosure and task chapters
GitHub / DagsHub	Private repositories with clean commits, DVC metadata, experiments and registry; student will grant grading access
Live API	Public health; key-protected POST /predict tested from a separate GitHub-hosted network
Local UI/history	Flask to Vercel integration active; original PostgreSQL rows and native model preserved
Evaluation integrity	Two locked model test passes, one per model; no later test reuse
Repository hygiene	No credentials, data/model binaries, agent files or decorative emoji in committed source
Bonus	Optional Grad-CAM not attempted

Before handing in, review this report, share both private repository links with the grader, and provide the API key through the instructor's private channel if authenticated prediction access is needed. Do not place it in the PDF. There is no instructor page limit currently specified.

Reproduction commands

```
.venv/bin/python scripts/check_disk.py
.venv/bin/dvc pull -j 2
.venv/bin/python scripts/check_repository.py --history
.venv/bin/python -m ruff check scripts visionguard tests docker cloud wsgi.py gunicorn.conf.py
.venv/bin/python -m pytest -q
sudo docker compose up -d --no-build --wait
```

dvc pull needs private access and sufficient space for missing objects; do not copy another full dataset on this constrained disk. CI builds on a fresh runner. The report builder consumes saved metrics and screenshots and performs no inference. The one-time final evaluator must not be rerun merely to regenerate figures. Consult docs/model-selection.md, docs/cloud-api.md and docs/cloud-integration.md for the model release and server-only settings.

References and evidence index

Assignment dataset: https://www.kaggle.com/datasets/cashbowman/ai-generated-images-vs-real-images

Keras transfer learning: https://keras.io/guides/transfer_learning/

MLflow TensorFlow tracking: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.tensorflow.html

DVC remote storage: https://doc.dvc.org/command-reference/remote/add

Docker Compose services: https://docs.docker.com/reference/compose-file/services/

GitHub Actions security: https://docs.github.com/en/actions/reference/security/secure-use

Vercel Hobby: https://vercel.com/docs/plans/hobby

Vercel function limits: https://vercel.com/docs/functions/limitations

Postman local collection runs: https://learning.postman.com/docs/postman-cli/postman-cli-run-collection

Detailed task-to-file mapping: docs/assignment-checklist.md. Machine-readable receipts: reports/data, reports/training, reports/selection, reports/evaluation, reports/registry, reports/application,

reports/container, reports/continuous-integration, reports/cloud, reports/integration and reports/labs. Figures are generated from those recorded results. Original command logs, database backups, secrets and runtime caches remain ignored.